CHAPTER 3 • TESTING AND CI ~15 Min

Student

We've done Git for solo work (Chapter 1) and Git + GitHub for team work (Chapter 2). What's next?

Claude • AI Assistant

Now we automate the safety. Every commit you make from Chapter 3 onward will be verified by machines - tests, linters, type checks, coverage - running on every push and every PR.

This chapter is the transition from 'careful engineer' to 'engineer whose care is enforced by tooling'. When it clicks (usually within one PR of using it), you will never willingly work on a codebase without it again.

WHAT CHAPTER 3 COVERS

Chapter 3 has six subsections. Two are **HIGH-EMPHASIS** (marked below) because they are daily-use skills for every professional engineer:

3.1 Why Test? - The ROI of Automated Testing. The compounding-returns case for testing, the bug-cost curve (1x -> 5000x by stage), the test pyramid and its ice-cream-cone anti-pattern, the AAA and FIRST principles, and your first pytest file for AskMyNotes' chunker.

3.2 pytest Advanced (HIGH-EMPHASIS). Fixtures (parameter injection, scopes, `yield` teardown, `conftest.py`), parametrize (one test many inputs with named cases and stacked-parametrize Cartesian products), marks (skip / xfail / custom marks for selective runs), and mocking (unittest.mock, mocker via pytest-mock, monkeypatch). Every real Python codebase uses these on every test file.

3.3 Coverage and the Test Pyramid. Line vs branch coverage, why 100% is a red flag, mutation testing as the gold standard for test quality, the coverage-quality 2x2 matrix, and how to wire coverage into CI with sensible targets (75-85% branch for production code).

3.4 What is CI/CD? The three flavors of CD (Continuous Integration, Continuous Delivery, Continuous Deployment), the standard pipeline shape (checkout -> setup -> lint -> type -> unit -> integration -> build -> publish), CI provider landscape (GitHub Actions, GitLab, CircleCI, Jenkins), safety patterns (canary, blue-green, feature flags, automated rollback), and the DORA golden signals (pipeline duration, flakiness, deploy frequency).

3.5 GitHub Actions (HIGH-EMPHASIS). The three-level structure (workflow -> jobs -> steps), triggers, marketplace actions, caching, matrix builds, secrets, parallel jobs with `needs:` sequencing, concurrency groups, timeouts, branch-protection integration, gh-CLI monitoring, and composite actions. AskMyNotes gets a real green CI badge by the end of this subsection.

3.6 Chapter 3 Review. 30-item vocabulary reference, 10-item self-check, the AskMyNotes upgrade exercise (real test suite + CI + coverage badges), and a full Chapter 3 command reference.

THE ASKMYNOTES UPGRADE

Student

What state does AskMyNotes end this chapter in?

Claude • AI Assistant

CONCEPT CARD

By the end of Chapter 3, AskMyNotes has:

- A `tests/` directory with **~15 tests** across chunker, loader, retrieval modules
- A `tests/conftest.py` with shared fixtures (sample*text, sample*pdf, mock*openai*client)
- **Parametrized tests** covering multiple chunk sizes and overlaps
- **Mocked LLM tests** that run in milliseconds without hitting OpenAI
- **Branch coverage enabled** in `pyproject.toml` with `--cov-fail-under=70`
- A **`.github/workflows/ci.yml`** with three jobs: lint (ruff + black), typecheck (mypy), test (pytest across Python 3.11 + 3.12 matrix)
- **`OPENAI_API_KEY` stored** as an encrypted GitHub secret
- **Branch protection** on `main` requiring all three checks + one approval + up-to-date branch
- **CI + coverage badges** on the README top

That combination is professional-grade quality automation. Any code reviewer, recruiter, or teacher looking at the repo will read the badges and infer 'this person knows what they're doing'. And you'll actually SLEEP better - the tests catch what your eyes miss.

REAL WORLD APPLICATION

This chapter's exact setup (pytest + coverage + GitHub Actions + branch protection) is the default configuration at essentially every professional Python codebase produced after ~2018. Learning it once on AskMyNotes means you're immediately productive on the first day of a real engineering job - you'll be reading and writing workflow files, adjusting coverage thresholds, and debugging failing CI runs like anyone else on the team.

⚠ WARNING

Chapter 3 has more code than any earlier chapter. Do the exercises hands-on - do not just read the code blocks.

The distance between 'I've read about pytest fixtures' and 'I confidently write pytest fixtures in a new codebase' is closed only by typing them yourself into a real project. AskMyNotes is your practice ground; use it heavily.

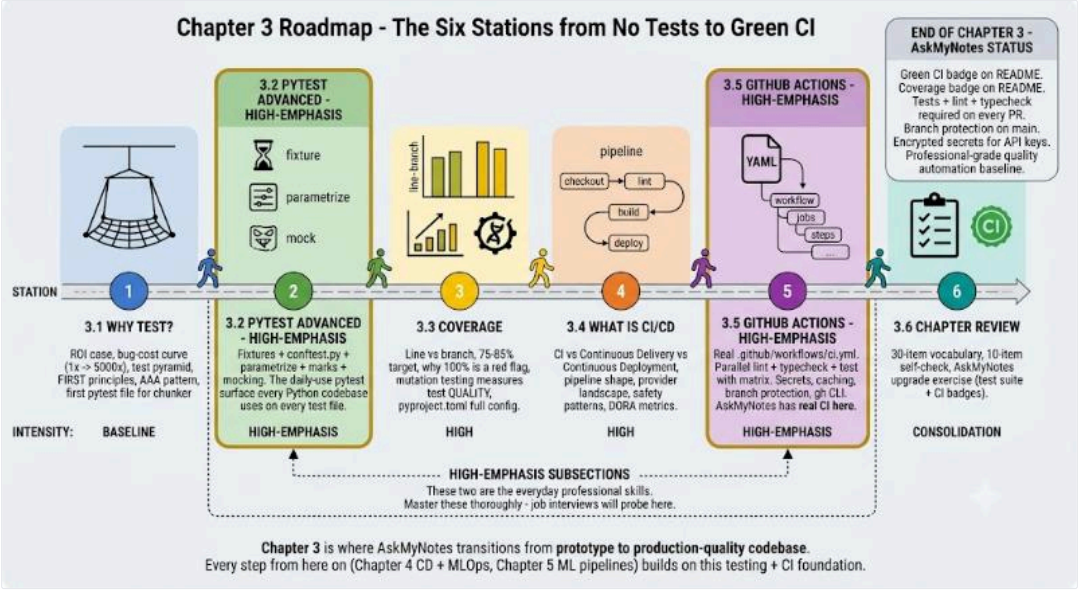
EXAM TIP CARD

Job interviews for software engineering roles will almost always include at least one testing question and one CI/CD question. The high-emphasis subsections (3.2 pytest advanced and 3.5 GitHub Actions) are the ones that give you a portfolio artifact you can point to during interviews - 'here's a real repo with fixtures, parametrized tests, mocking, and a CI workflow I built'.

That's stronger than any answer to 'do you know pytest?'.

THE CHAPTER 3 ROADMAP

Six stations of the Chapter 3 journey - see the roadmap figure below. Each subsection is roughly 2 hours reading + hands-on. High-emphasis subsections (3.2, 3.5) are longer and denser; budget accordingly.



Chapter 3 Roadmap - The Six Stations from No Tests to Green CI

Section 3 At-a-Glance - What Each Subsection Covers

Sub	Title	Focus	Emphasis
3.1	Why Test?	ROI case, bug-cost curve, test pyramid, FIRST principles, first pytest file	Foundation
3.2	pytest Advanced	Fixtures, parametrize, marks, mocking - the daily-use pytest surface	HIGH-EMPHASIS
3.3	Coverage & Pyramid	Line vs branch coverage, 75-85% target, mutation testing, full pyproject.toml config	High
3.4	What is CI/CD?	CI vs Delivery vs Deployment, pipeline stages, providers, safety patterns, DORA metrics	High
3.5	GitHub Actions	Workflow YAML, parallel jobs, matrix, secrets, caching, branch protection - real ci.yml for AskMyNotes	HIGH-EMPHASIS
3.6	Chapter Review	30-item vocab, 10-item self-check, AskMyNotes CI upgrade exercise	Consolidation

```
bash — Section 3 preview - the testing + CI toolbox you'll master
```

```
# ---- pytest fluency (3.1, 3.2) ----
pytest -v                           # run tests verbosely
pytest -k "chunker" -v              # filter by name
pytest -m "not slow" -v             # filter by mark
pytest --lf -x                      # rerun last failed, stop at first
pytest --durations=10               # 10 slowest tests

# ---- Coverage (3.3) ----
pytest --cov=askmynotes --cov-branch --cov-report=term-missing
pytest --cov=askmynotes --cov-fail-under=80    # CI threshold

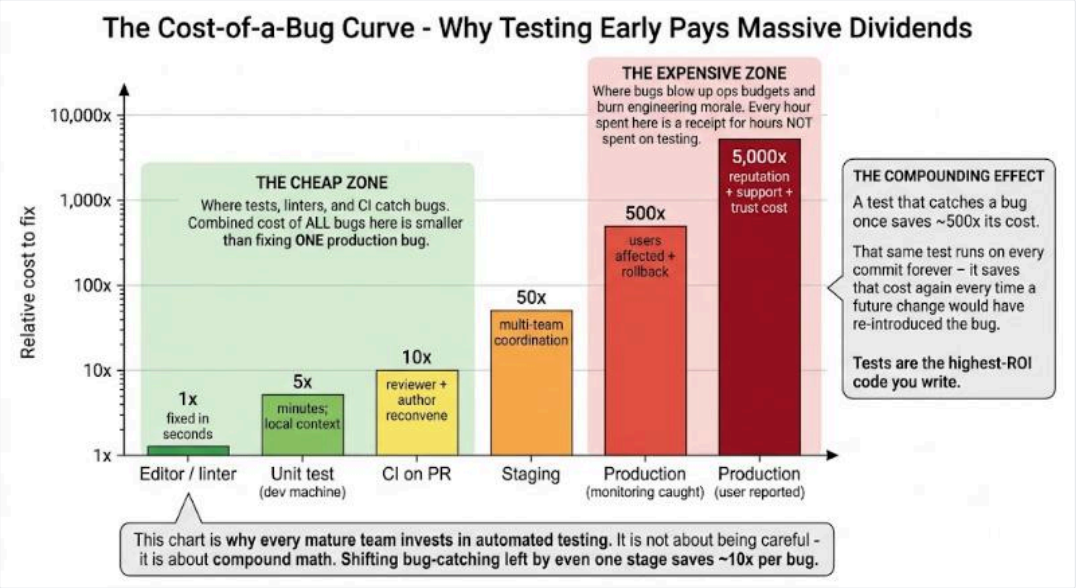
# ---- Static checks (3.4 preview, 4.1 depth) ----
ruff check .                        # lint
black --check .                     # format check
mypy askmynotes/                    # type check

# ---- GitHub Actions (3.5) via gh CLI ----
gh workflow list                    # workflows in repo
gh run list                         # recent CI runs
gh run watch                        # tail latest live
gh run view --log-failed            # only failed logs
gh secret set OPENAI_API_KEY        # add encrypted secret

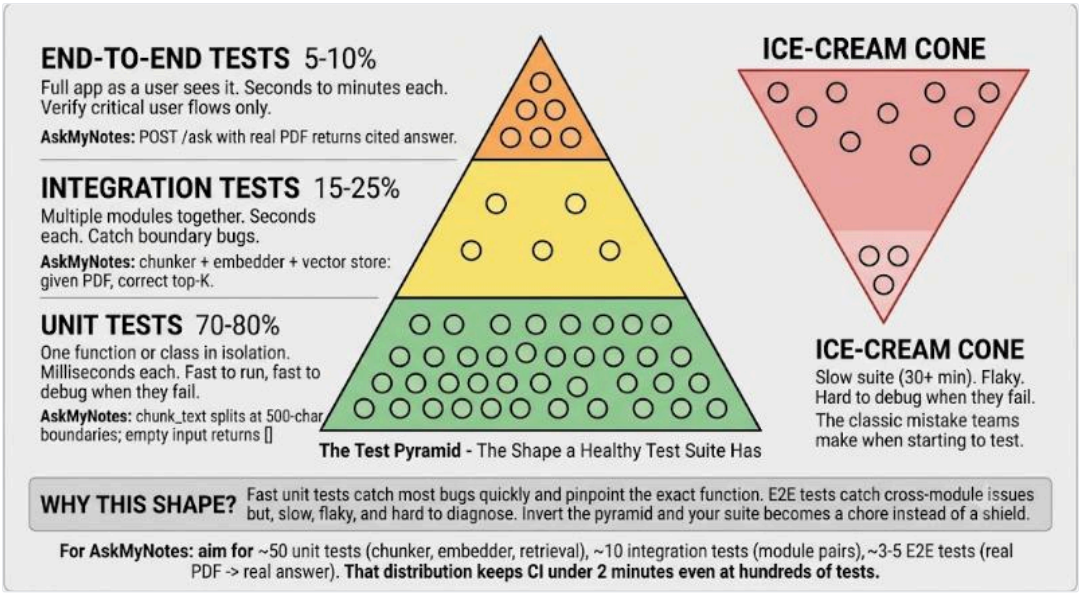
# ---- Files you will create in Chapter 3 ----
#  tests/conftest.py                shared fixtures
#  tests/test_chunker.py            parametrized tests
#  tests/test_retrieval.py          mocked LLM tests
#  pyproject.toml                   pytest + coverage config
#  .github/workflows/ci.yml         real CI pipeline
#  README.md                        add CI + coverage badges
```

WHY TEST? - THE ROI OF AUTOMATED TESTING

Claude · AI Assistant



The Cost-of-a-Bug Curve - Why Testing Early Pays Massive Dividends



The Test Pyramid - The Shape a Healthy Test Suite Has

python — First-day test file for AskMyNotes chunker - the shape to copy

```python
# tests/test_chunker.py
# Convention: test files named test_*.py; test functions named test_*.
# pytest auto-discovers both.

from askmynotes.chunker import chunk_text


# --- Empty / edge cases first (the boring but crucial ones) ---

def test_empty_string_returns_empty_list():
    assert chunk_text('') == []

def test_whitespace_only_input_returns_expected_shape():
    # Design decision: whitespace-only is still 'text', return one chunk
    result = chunk_text('   \n\n\t  ', size=500)
    assert len(result) == 1


# --- Happy-path cases (the normal usage) ---

def test_short_text_returns_single_chunk_equal_to_input():
    text = 'Hello world'
    result = chunk_text(text, size=500)
    assert result == ['Hello world']

def test_exact_boundary_returns_single_chunk():
    text = 'x' * 500
    result = chunk_text(text, size=500, overlap=50)
    assert len(result) == 1
    assert result[0] == 'x' * 500


# --- Multi-chunk cases (the ones the chunker exists to handle) ---

def test_1200_char_text_splits_into_three_500_char_chunks():
    text = 'x' * 1200
    chunks = chunk_text(text, size=500, overlap=50)
    # step = size - overlap = 450
    # chunk boundaries: 0-500, 450-950, 900-1200
    assert len(chunks) == 3
    assert all(len(c) <= 500 for c in chunks)

def test_chunks_overlap_by_configured_amount():
    text = 'abcdefghij' * 100                        # 1000 chars
```

```
    chunks = chunk_text(text, size=200, overlap=50)
    # Last 50 chars of chunk 0 should equal first 50 chars of chunk 1
    assert chunks[0][-50:] == chunks[1][:50]


# --- Run the suite ---
# pytest -v                           (verbose: shows each test name)
# pytest -v tests/test_chunker.py     (just this file)
# pytest -k "boundary"                (only tests matching 'boundary')
# pytest --tb=short                   (shorter tracebacks)
# pytest -x                           (stop on first failure)
```

Student

I write code that works. Why do I need to write ADDITIONAL code that just checks the first code? Feels like doubling my work.

Claude • AI Assistant

It feels that way at first, then it doesn't. Automated tests are one of the highest-return investments in software engineering - they compound the way a savings account compounds.

This subsection walks through the concrete ROI (what tests actually pay back, in time saved), the different kinds of tests you'll write, and the mindset shift that separates 'programmers who test' from 'programmers who don't understand why testing exists'. By the end of this subsection you'll want to write tests, not tolerate them.

THE FIVE THINGS TESTS PAY YOU BACK

Student

Give me the concrete returns.

Claude • AI Assistant

CONCEPT CARD

Five compounding benefits, in rough order of visibility:

1. Catch regressions cheaply. A regression is a bug you re-introduce because you forgot the original edge case.

Every bug you've ever fixed is a regression waiting to happen the next time someone touches that code. A test written when you first fix a bug is a permanent guard - a two-line assertion prevents the same bug from ever returning silently.

2. Refactor without fear. Renaming a function, restructuring modules, swapping a library - all become safe when a green test suite tells you the behavior is unchanged.

Without tests, every refactor is a coin flip. With tests, refactoring becomes a routine cleanup habit instead of a scary project.

3. Enable continuous integration. CI is only useful if the tests are meaningful. A repo with strong tests + green CI ships changes multiple times a day; a repo with no tests limps along at one release per quarter, half the time in rollback.

4. Serve as executable documentation. A test is the truest documentation of how a function is supposed to behave, because it either passes or fails - text-only documentation goes stale silently.

New engineers reading your test file learn 'oh, this function should raise `ValueError` on empty input, and return `[]` on whitespace-only input'. No amount of README could be as precise.

5. Force better design. Untestable code is usually badly-designed code: too many dependencies, doing too many things, hidden state everywhere.

Writing tests forces you to make code testable - which usually means smaller functions, explicit dependencies, and clearer boundaries. The design pressure from testing is often more valuable than the tests themselves.

THE COST-OF-A-BUG CURVE

Student

You said tests are cheap. How cheap compared to shipping the bug?

Claude • AI Assistant

CONCEPT CARD

Every study of software defect economics shows the same pattern: **the later you catch a bug, the more it costs to fix, by roughly an order of magnitude per stage**. A bug caught while the developer is still typing costs about 1x (fix in seconds).

Caught by a local test suite: ~5x. By CI on the PR: ~10x.

By staging: ~50x. In production by monitoring: ~500x.

In production by a user: ~5000x - counting rollback effort, postmortem, support burden, and reputational cost.

A 30-second unit test is cheaper than one 2AM production page. And the test runs on every commit forever - the ratio only widens with time.

THE TEST PYRAMID

Student

What kinds of tests exist?

Claude • AI Assistant

CONCEPT CARD

The classic **test pyramid** (from Mike Cohn) categorizes tests by scope and cost:

- **Unit tests** (base of pyramid, most numerous) - one function or class in isolation. Milliseconds each. Should make up 70-80% of your test count. AskMyNotes example: `chunk_text('...', size=500)` returns chunks of the right shape.
- **Integration tests** (middle) - multiple modules working together. Seconds each. Roughly 15-25% of tests. AskMyNotes example: chunker + embedder + vector store, when given a PDF, produce searchable embeddings that return correct top-K.
- **End-to-end tests** (top, fewest) - the full app as a user experiences it. Seconds to minutes each. 5-10% of tests. AskMyNotes example: POST `/ask` with a real question against a real PDF returns an answer that mentions the correct source page.

Why a pyramid? Fast unit tests catch most bugs quickly; slower integration/E2E tests catch the smaller number of issues that only appear across module boundaries.

Inverting the pyramid (mostly E2E, few unit tests) gives you a slow, flaky, hard-to-debug suite - the classic anti-pattern called the **ice-cream cone**.

THE MINDSET SHIFT

Student

How do people who love testing actually think about it?

Claude • AI Assistant

CONCEPT CARD

Three internal shifts:

Shift 1: Tests are cheaper than debugging. A senior engineer, presented with a mystery bug, will often first write a test that reproduces the bug (making it a red test), then fix the code (making it green). The test takes 5 minutes to write and permanently prevents regression.

Compared to hours of print-debugging with no permanent artifact, it's a bargain.

Shift 2: 'It works on my machine' is not a done state. Done means 'a test proves it works, that test is committed, and CI runs it on every PR forever'. Anything less is 'hopefully still works, until we discover otherwise'.

Shift 3: The test suite is a product asset. A repo's test suite is often more valuable than its production code - because production code can be rewritten, but the tests encode all the edge cases and requirements the team ever discovered. Treat tests with the same rigor as production code: clear names, no duplication, focused assertions.

WHAT GOOD TESTS LOOK LIKE

Student

Show me a good test.

Claude • AI Assistant

EXAMPLE CARD

The chunker in AskMyNotes:

python — Code Example

```python
# askmynotes/chunker.py
def chunk_text(text: str, size: int = 500, overlap: int = 50) -> list[str]:
    """Split text into overlapping chunks of ~size characters."""
    if not text:
        return []
    step = size - overlap
    return [text[i:i+size] for i in range(0, len(text), step)]
```

A well-structured test file:

python — Code Example

```python
# tests/test_chunker.py
from askmynotes.chunker import chunk_text

def test_empty_input_returns_empty_list():
    assert chunk_text('') == []

def test_short_input_returns_one_chunk():
    chunks = chunk_text('Hello world', size=500)
    assert len(chunks) == 1
    assert chunks[0] == 'Hello world'

def test_long_input_is_split_into_multiple_chunks():
    text = 'x' * 1200
    chunks = chunk_text(text, size=500, overlap=50)
    assert len(chunks) == 3       # 0-500, 450-950, 900-1200
    assert all(len(c) <= 500 for c in chunks)

def test_chunks_overlap_by_configured_amount():
    text = 'abcdefghij' * 100     # 1000 chars
    chunks = chunk_text(text, size=200, overlap=50)
    # Last 50 chars of chunk 0 should be first 50 chars of chunk 1
    assert chunks[0][-50:] == chunks[1][:50]
```

Notice: each test has ONE responsibility, a **name that describes the behavior** (not 'test*chunker*1'), and asserts something specific. This is the AAA pattern (**Arrange -> Act -> Assert**) even without labels: set up input, call the function, check the output.

THE FIRST PRINCIPLES

Student

Any rules of thumb for good tests?

Claude • AI Assistant

CONCEPT CARD

The FIRST acronym (from *Clean Code*):

- **F**ast - tests should run in milliseconds; slow tests get skipped
- **I**ndependent - order of tests should not matter; no shared mutable state

- **R**epeatable - same result every time; no random inputs without seeds, no reliance on wall-clock time
- **S**elf-validating - a test passes or fails, no 'inspect the output manually'
- **T**imely - written when the code is written, not 'later' (which usually means never)

When a test violates FIRST, it becomes a burden. Flaky tests (violating R) train the team to ignore red CI.

Order-dependent tests (violating I) hide real bugs. Slow tests get skipped by developers under deadline pressure.

Every one of these is a well-known anti-pattern for a reason.

ANALOGY

Testing is like the **safety net under a trapeze artist**. Without it, one small mistake ends the career (production disaster).

With it, the artist takes bigger risks, tries new tricks (refactors freely), and recovers gracefully. The net doesn't slow the performance - it enables the performance.

Same for tests: they don't slow development, they enable the rapid, confident development that separates fast teams from careful-but-slow ones.

REAL WORLD APPLICATION

A team of 8 engineers at a healthtech startup had zero automated tests for their first 18 months. They shipped features fast but spent 60% of their time firefighting production bugs.

They stopped feature work for 3 weeks to backfill a test suite covering the top 50 code paths. Six months later: bug reports down 70%, feature velocity up 40%, on-call rotation went from dreaded to routine.

The 'wasted' 3 weeks paid back within 2 months.

WARNING

Don't confuse 'high coverage' with 'well-tested'. A test suite can hit 95% coverage while asserting almost nothing - just importing modules and calling functions without checking outputs.

Coverage measures which lines RAN; it does not measure whether behavior was verified. Chapter 3.3 covers this in detail.

COMMON PITFALL / MISTAKE

Writing tests that duplicate the implementation. `assert chunk_text('abc', 500) == [text[0:500] for i in range(0, len(text), 450)]` is not a test - it just re-runs the implementation logic in a different form.

Good tests express the *expected behavior* in the simplest terms a human can verify, not the internal algorithm.

EXAM TIP CARD

Common interview question: 'Why do you write tests?' Weak answer: 'to catch bugs'. Strong answer: 'Five reasons - catch regressions cheaply, enable safe refactoring, enable CI/CD, act as executable documentation, and force better design by exposing hidden dependencies.

The ROI compounds because tests run automatically forever.'

PRACTICE QUIZ

Your team has 100 tests: 5 unit tests, 20 integration tests, 75 E2E tests taking 45 minutes total. What's the biggest problem?

A) 100 tests is too few; add more

B) The pyramid is inverted (ice-cream cone) - too few unit tests, too many E2E

C) 45 minutes is fine; nothing to fix

D) Integration tests are unnecessary

▼ Reveal Correct Answer

Answer: B. This is the classic ice-cream cone anti-pattern. Rebalance toward unit tests: fast, focused, easy to debug. Keep just enough E2E tests to verify critical user flows.

EXERCISE

On AskMyNotes: (a) create `tests/test_chunker.py`, (b) write 5 tests covering empty input, whitespace-only, short text (< chunk size), exact-boundary text (== chunk size), and long text (multiple chunks), (c) run `pytest -v` and see all 5 pass, (d) deliberately introduce a bug (change `size - overlap` to `size + overlap`), (e) rerun `pytest` and watch the specific tests turn red with clear failure messages.

AI DEEP DIVE PROMPT

> Ask an AI: *'Give me the ROI case for automated testing to a skeptical junior developer who says "my code works, I don't need tests". Cover the cost-of-a-bug curve, the five compounding returns, and one story where lack of tests caused a real incident.'*

SUMMARY CARD

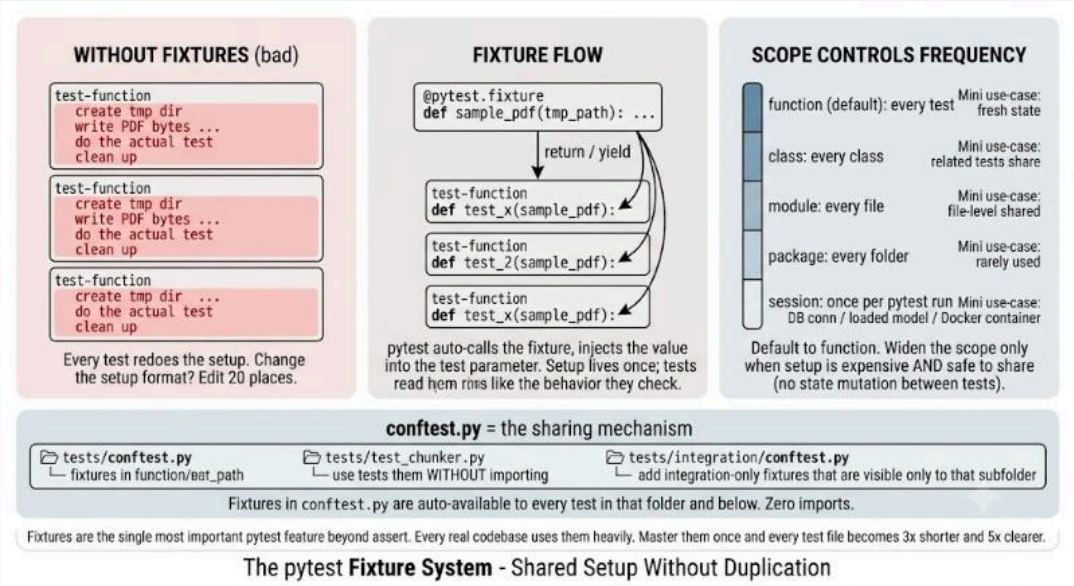
Tests are one of the highest-ROI investments in engineering. Five compounding returns: **catch regressions cheaply, enable fearless refactoring, enable CI/CD, act as executable documentation, force better design**.

Bug-cost curve rises by ~10x per stage - a test that runs on every commit is orders of magnitude cheaper than a 2AM production page. Test types stack in a **pyramid**: many fast unit tests at the base, some integration tests in the middle, few E2E tests at the top.

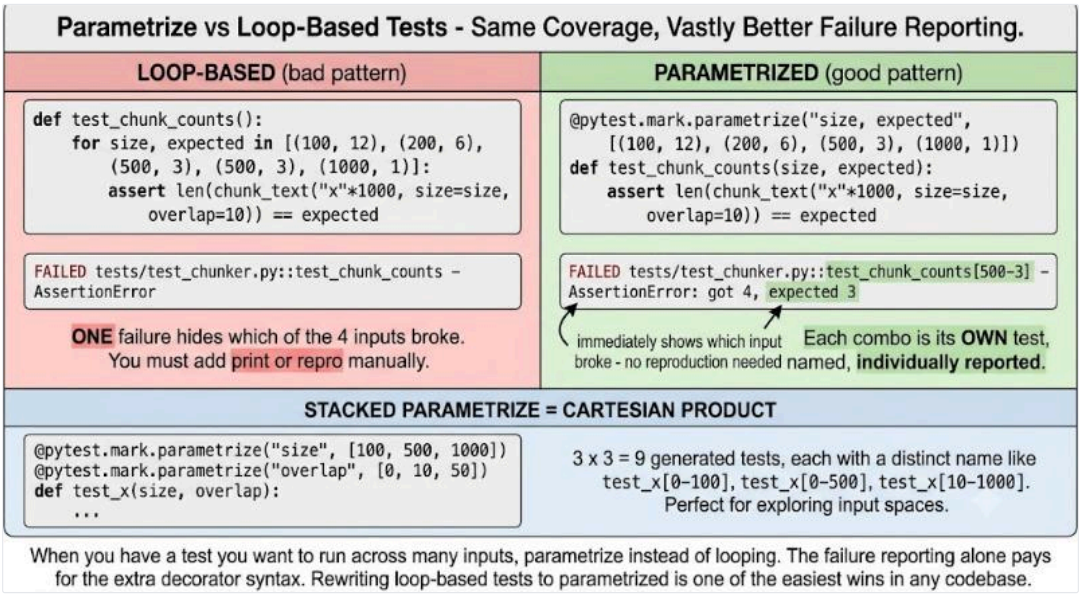
Good tests follow **FIRST**: Fast, Independent, Repeatable, Self-validating, Timely. Coverage measures which lines ran, not whether behavior was checked - don't confuse the two.

PYTEST ADVANCED - FIXTURES, PARAMETRIZE, MOCKING

Claude · AI Assistant



The pytest Fixture System - Shared Setup Without Duplication



Parametrize vs Loop-Based Tests - Same Coverage, Vastly Better Failure Reporting

```
python — conftest.py - the shared fixture file that powers your test suite
```

```python
# tests/conftest.py
# Auto-discovered by pytest. Fixtures defined here are available in
# every test file under this folder - NO IMPORTS needed.

import pytest
from pathlib import Path
from unittest.mock import MagicMock


# --- Function-scoped: fresh state per test (default) ---

@pytest.fixture
def sample_text() -> str:
    """A reproducible chunk of text for chunker tests."""
    return 'The quick brown fox jumps over the lazy dog. ' * 40


@pytest.fixture
def sample_pdf(tmp_path: Path) -> Path:
    """Create a minimal PDF file in a fresh temp dir per test."""
    pdf = tmp_path / 'sample.pdf'
    pdf.write_bytes(
        b'%PDF-1.4\n1 0 obj<</Type/Catalog>>endobj\ntrailer<<>>\n%%EOF'
    )
    return pdf


@pytest.fixture
def mock_openai_client(mocker):
    """Return a mocked OpenAI client whose responses can be programmed."""
    client = MagicMock()
    client.chat.completions.create.return_value = MagicMock(
        choices=[MagicMock(message=MagicMock(content='Mocked answer.'))]
    )
    mocker.patch('askmynotes.retrieval.client', client)
    return client


# --- Session-scoped: expensive resources loaded once for the whole run ---

@pytest.fixture(scope='session')
def embedding_model():
    """Load the embedding model exactly once for the whole test run."""
    from sentence_transformers import SentenceTransformer
    print('Loading embedding model (once per session)...')
    return SentenceTransformer('all-MiniLM-L6-v2')


# --- Yield fixture: setup + teardown ---

@pytest.fixture
def vector_store(tmp_path):
    from askmynotes.vector_store import VectorStore
    store = VectorStore(path=tmp_path / 'idx')       # setup
    store.initialize()

    yield store                                      # test runs here

    store.close()                                    # teardown (always runs)
```

python — Real parametrized + mocked test file for AskMyNotes

```python
# tests/test_retrieval.py
import pytest
from askmynotes import retrieval
```

```python
# --- Parametrized: one test, many inputs, individual failure reporting ---

@pytest.mark.parametrize(
    'question, context, expected_word',
    [
        ('What is the capital?', ['The capital is Delhi.'], 'Delhi'),
        ('Who wrote it?',        ['Written by Rao.'],       'Rao'),
        ('When founded?',        ['Founded in 1972.'],      '1972'),
    ],
    ids=['capital', 'author', 'year'],               # named test cases
)
def test_answer_question_returns_context_relevant_answer(
    mock_openai_client, question, context, expected_word
):
    # Program the mock to echo the expected word for this parametrized case
    mock_openai_client.chat.completions.create.return_value.choices[0]\
        .message.content = f'The answer contains {expected_word}.'

    result = retrieval.answer_question(question=question, context=context)

    assert expected_word in result
    # Verify the LLM was called and question was in the prompt
    mock_openai_client.chat.completions.create.assert_called_once()
    call = mock_openai_client.chat.completions.create.call_args
    prompt = call.kwargs['messages'][0]['content']
    assert question in prompt


# --- Marks: exclude from fast CI, run in nightly ---

@pytest.mark.slow
@pytest.mark.integration
def test_end_to_end_with_real_pdf(sample_pdf, embedding_model):
    from askmynotes.pipeline import ingest_and_answer
    answer = ingest_and_answer(sample_pdf, 'What is this document?')
    assert isinstance(answer, str)
    assert len(answer) > 0


# --- Skip / xfail: express known conditions ---

@pytest.mark.skip(reason='awaiting pdf-tables release')
def test_extracts_tables_from_pdf(): ...


@pytest.mark.xfail(reason='known: fails on scanned PDFs without OCR')
def test_scanned_pdf_returns_text(sample_pdf): ...


# ---- Run these ----
# pytest -v                             # all tests, verbose
# pytest -v -m 'not slow'               # exclude slow ones (fast dev loop)
# pytest -v -m integration              # only integration tests
# pytest -v -k 'capital'                # only the parametrized case named 'capital'
# pytest --lf -x                        # rerun last failures, stop at first
```

Student

I can write basic pytest assertions. But real teams' test files have `@pytest.fixture` and `@pytest.mark.parametrize` and `MagicMock` all over the place. What is all that, and why?

Claude · AI Assistant

Because plain `assert` gets you 80% of the way, but the remaining 20% - **shared setup, running the same test across many inputs, isolating code from external dependencies** - is where beginner test suites become bloated, slow, or unreliable. pytest's advanced features (fixtures, parametrize, marks, mocking) exist to make that last 20% clean and fast.

This subsection is one of Chapter 3's two high-emphasis subsections because these are the tools you'll use on every real codebase from day one of your career.

FIXTURES - SHARED SETUP DONE RIGHT

Student

What's a fixture?

Claude · AI Assistant

CONCEPT CARD

A **fixture** is a function pytest calls automatically to provide inputs or setup to your tests. The classic use: you need a temporary PDF file for several chunker tests.

Without fixtures, each test would set one up and tear it down manually - duplicated code everywhere. With fixtures:

python — Code Example

```python
import pytest
from pathlib import Path

@pytest.fixture
def sample_pdf(tmp_path: Path) -> Path:
    """Create a small test PDF and return its path."""
    pdf = tmp_path / 'sample.pdf'
    pdf.write_bytes(b'%PDF-1.4\n... minimal PDF bytes ...')
    return pdf

def test_pdf_loader_reads_bytes(sample_pdf):
    assert sample_pdf.exists()
    assert sample_pdf.read_bytes().startswith(b'%PDF')

def test_pdf_loader_returns_text(sample_pdf):
    from askmynotes.loader import load_pdf
    text = load_pdf(sample_pdf)
    assert isinstance(text, str)
```

Two things happen automatically:

1. **Naming a fixture as a parameter** (`def test_pdf_loader_reads_bytes(sample_pdf)`) tells pytest 'call the `sample_pdf` fixture and pass its return value into this parameter'.
2. `tmp_path` is a **built-in pytest fixture** giving each test its own fresh temp directory - auto-cleaned after the test. There are dozens of built-in fixtures: `tmp_path` , `monkeypatch` , `capsys` , `caplog` , `request` .

The result: setup code lives in one place; tests read like the behavior they check, not like plumbing.

FIXTURE SCOPE - HOW OFTEN TO RUN SETUP

Student

What if the setup is expensive?

Claude · AI Assistant

CONCEPT CARD

Fixture **scope** controls how often it runs:

python — Code Example

```python
# Runs once PER TEST (default) - fresh state, no leakage
@pytest.fixture
def clean_index():
    return VectorIndex()

# Runs once PER TEST FILE - good for expensive shared setup within a file
@pytest.fixture(scope='module')
def loaded_model():
    print('Loading embedding model...')
    return SentenceTransformer('all-MiniLM-L6-v2')

# Runs once PER TEST SESSION - the entire pytest run
@pytest.fixture(scope='session')
def postgres_container():
    container = start_test_postgres()
    yield container
    container.stop()
```

Rule of thumb: use function scope (default) unless the fixture is genuinely expensive AND safe to share. A shared-scope fixture that mutates state between tests will cause flaky failures - one test contaminates the next.

When in doubt, function-scope; profile later if it matters.

YIELD FOR SETUP + TEARDOWN

Student

How do I clean up after?

Claude • AI Assistant

CONCEPT CARD

Use `yield` inside a fixture. Code before `yield` = setup; code after = teardown:

python — Code Example

```python
@pytest.fixture
def vector_store(tmp_path):
    # Setup: create a fresh vector store on disk
    store = VectorStore(path=tmp_path / 'index')
    store.initialize()

    yield store                                # test runs with this object

    # Teardown: runs after the test, even if the test fails
    store.close()
    # tmp_path is auto-cleaned by pytest
```

Guaranteed teardown after the test regardless of success or failure - the same shape as `try / finally`.

CONFTEST.PY - FIXTURES SHARED ACROSS FILES

Student

What if two test files need the same fixture?

Claude • AI Assistant

CONCEPT CARD

Put it in `conftest.py`. This file is auto-discovered by pytest; any fixtures defined here are available in every test file in the same directory (and below). No imports needed.

code — Code Example

```
askmynotes/
  chunker.py
  loader.py
tests/
  conftest.py          <-- shared fixtures live here
  test_chunker.py      <-- uses fixtures from conftest.py without importing
  test_loader.py
  integration/
    conftest.py        <-- integration-only fixtures
    test_end_to_end.py
```

The nesting matters: a fixture in `tests/conftest.py` is available to everything under `tests/`; a fixture in `tests/integration/conftest.py` is only available to tests in `tests/integration/`. This is how large codebases keep their fixture scopes tidy.

PARAMETRIZE - ONE TEST, MANY INPUTS

Student

What about running the same test with many inputs?

Claude • AI Assistant

CONCEPT CARD

Enter `@pytest.mark.parametrize`. Instead of:

python — Code Example

```python
def test_chunk_size_100(): assert len(chunk_text('x'*1000, size=100)) == 12
def test_chunk_size_200(): assert len(chunk_text('x'*1000, size=200)) == 6
def test_chunk_size_500(): assert len(chunk_text('x'*1000, size=500)) == 3
```

Write one test with many inputs:

python — Code Example

```python
@pytest.mark.parametrize(
    'size, expected_chunk_count',
    [
        (100, 12),
        (200, 6),
        (500, 3),
        (1000, 1),
    ],
)
def test_chunk_count_for_1000_chars(size, expected_chunk_count):
    text = 'x' * 1000
    assert len(chunk_text(text, size=size, overlap=10)) == expected_chunk_count
```

pytest generates 4 tests behind the scenes, each named clearly: `test_chunk_count_for_1000_chars[100-12]`, `[200-6]`, `[500-3]`, `[1000-1]`. When one fails, the failure shows which specific parameters broke - vastly easier to debug than one giant loop-based test.

You can stack `@parametrize` decorators to get the Cartesian product:

python — Code Example

```python
@pytest.mark.parametrize('size', [100, 500, 1000])
@pytest.mark.parametrize('overlap', [0, 10, 50])
def test_chunker_never_loses_text(size, overlap):
    text = 'x' * 2000
    chunks = chunk_text(text, size=size, overlap=overlap)
    # Property: joining chunks with overlap-aware dedup recovers the original
    joined = chunks[0] + ''.join(c[overlap:] for c in chunks[1:])
    assert joined == text
```

3 sizes x 3 overlaps = 9 generated tests, each independently named and reported.

MARKS - SELECTIVE TEST RUNS

Student

What are marks for?

Claude · AI Assistant

CONCEPT CARD

`@pytest.mark.<name>` attaches a label to a test. Two big uses:

Built-in marks:

python — Code Example

```python
@pytest.mark.skip(reason='PDF library upgrade pending')
def test_pdf_with_images(): ...

@pytest.mark.skipif(sys.version_info < (3, 10), reason='needs Python 3.10+')
def test_new_syntax(): ...

@pytest.mark.xfail(reason='known bug, fix pending')
def test_edge_case_broken(): ...
```

Custom marks for selective runs:

python — Code Example

```python
@pytest.mark.slow
def test_full_pipeline_on_large_pdf(): ...

@pytest.mark.integration
def test_chunker_plus_embedder_plus_vectorstore(): ...
```

Register them in `pytest.ini` (avoids warnings):

ini — Code Example

```ini
[pytest]
markers =
    slow: marks tests as slow (>1s)
    integration: marks tests as integration (need external services)
```

Then run subsets:

bash — Code Example

```bash
pytest -m 'not slow'                # skip slow tests in normal dev loop
pytest -m integration               # only integration tests
pytest -m 'slow or integration'     # both
```

Common CI setup: fast tests on every PR; slow + integration tests nightly.

MOCKING - ISOLATING CODE FROM EXTERNAL DEPENDENCIES

Student

What about code that calls an LLM or hits a database?

Claude · AI Assistant

CONCEPT CARD

Mocking replaces real dependencies with fake objects that return canned responses. Purpose: keep tests fast (no network), deterministic (no LLM randomness), and free (no API costs).

`unittest.mock` is built-in; pytest has `mocker` (via `pytest-mock`) as a friendlier fixture wrapper.

python — Code Example

```python
# askmynotes/retrieval.py
from openai import OpenAI

client = OpenAI()

def answer_question(question: str, context: list[str]) -> str:
    prompt = f"Context: {context}\n\nQuestion: {question}"
    response = client.chat.completions.create(
        model='gpt-4o-mini',
        messages=[{'role': 'user', 'content': prompt}],
    )
    return response.choices[0].message.content
```

Testing without hitting OpenAI:

python — Code Example

```python
# tests/test_retrieval.py
from unittest.mock import MagicMock
from askmynotes import retrieval

def test_answer_question_uses_context(mocker):
    # Arrange: mock the OpenAI client's create() call
    fake_response = MagicMock()
    fake_response.choices[0].message.content = 'The capital is Delhi.'
    mocker.patch.object(
        retrieval.client.chat.completions,
        'create',
        return_value=fake_response,
    )

    # Act
    result = retrieval.answer_question(
        question='What is the capital of India?',
        context=['India is a country in South Asia. Its capital is New Delhi.'],
    )

    # Assert
    assert result == 'The capital is Delhi.'
    retrieval.client.chat.completions.create.assert_called_once()
    call_args = retrieval.client.chat.completions.create.call_args
    assert 'What is the capital of India?' in call_args.kwargs['messages'][0]['content']
```

This test runs in milliseconds, costs nothing, and verifies the important behavior: **the question is passed in the prompt, and the model's response is returned**. Whether the real OpenAI returns 'Delhi' is not our test's concern - that's OpenAI's problem.

MONKEYPATCH - THE BUILT-IN PATCHER

Student

Do I need `pytest-mock` ?

Claude • AI Assistant

CONCEPT CARD

For simple env-var or function replacements, pytest's built-in `monkeypatch` fixture does the job with no extra dependency:

python — Code Example

```python
def test_uses_api_key_from_env(monkeypatch):
    monkeypatch.setenv('OPENAI_API_KEY', 'test-key-12345')
    from askmynotes.config import get_api_key
    assert get_api_key() == 'test-key-12345'
    # env var is auto-restored after the test

def test_uses_stubbed_time(monkeypatch):
    monkeypatch.setattr('time.time', lambda: 1700000000.0)
    from askmynotes.metrics import current_bucket
    assert current_bucket() == 1700000000 // 60
```

For patching methods and asserting on calls, `mocker` (from `pytest-mock`) is cleaner - it wraps `unittest.mock.patch` as a fixture that auto-cleans.

ANALOGY

Fixtures are like the **prep work in a professional kitchen**: mise en place - chopped vegetables, measured stock, hot pan ready. When the actual dish is cooked (the test runs), everything is at hand and the cook focuses on the cooking.

Without prep, every dish redoes the chopping. Parametrize is like a **taste test with many salt levels**: same recipe, small dial change, compare outcomes side by side.

Mocking is a **rehearsal dinner** - all the moves without the risk or cost of the real event.

REAL WORLD APPLICATION

A team's LLM-heavy API test suite was costing
400/monthinOpenAIcharges – becauseeveryP RranthefulltestsuitehittingtherealAPI.Switchingtomockedresponses(realAPIcalledonlyinanightlyjob)droppeditto
8/month AND cut CI runtime from 12 minutes to 90 seconds.

That's the mock ROI in one sentence.

WARNING

Do NOT over-mock. If you mock every dependency, your tests verify only that mocked objects were called - not that real code works.

Balance: mock **network, database, external APIs, non-deterministic inputs**. Don't mock **your own pure functions**, **standard library**, or **anything in the same module**.

When in doubt, ask 'is this mock hiding a real bug the test should catch?'

COMMON PITFALL / MISTAKE

Mocking the wrong path. `mocker.patch('openai.OpenAI')` patches openai at its source; `mocker.patch('askmynotes.retrieval.OpenAI')` patches it where it's *used*.

If your production code does `from openai import OpenAI`, you must patch the second path (where the name is bound), not the first. This 'where to patch' rule catches almost every mock-related mystery.

EXAM TIP CARD

Star interview question: **'How do you test a function that calls an external API?'** Structured answer: '(1) Inject the client or use dependency injection so the API is easily replaceable, (2) In tests, use `mocker.patch` or `monkeypatch` to replace the API call with a fake, (3) Assert both on the returned value AND on how the mock was called (right prompt, right params), (4) Keep a small number of *real* integration tests behind a mark like `@pytest.mark.integration` for a nightly job.' That framing signals senior thinking.

PRACTICE QUIZ

You have 20 tests that each need a loaded embedding model (2 seconds to instantiate). What's the cleanest fix?

A) Load the model at the top of each test function

B) Load the model in a `conftest.py` fixture with `scope='session'`

C) Load the model globally at module import time

D) Skip the tests and hope for the best

▼ Reveal Correct Answer

Answer: B. Session-scoped fixture in conftest.py loads once for the whole test run, is available in every test file, and is auto-cleaned. A wastes 40 seconds. C causes issues with test isolation and slow imports. D is not a serious option.

EXERCISE

On AskMyNotes: (a) create `tests/conftest.py` with a `sample_text` fixture returning a 1500-char string, (b) rewrite one existing chunker test to use the fixture, (c) add a parametrized test that runs across `size = [100, 500, 1000]` and asserts chunk count matches an expected formula, (d) add a test for `answer_question` that mocks the LLM client and asserts the mock was called with the user's question in the prompt.

AI DEEP DIVE PROMPT

Ask an AI: *'Rewrite this pytest file to use fixtures, parametrize, and mocking best practices. For each change, explain why the new version is better than the old one. Point out any anti-patterns in my current tests.'*

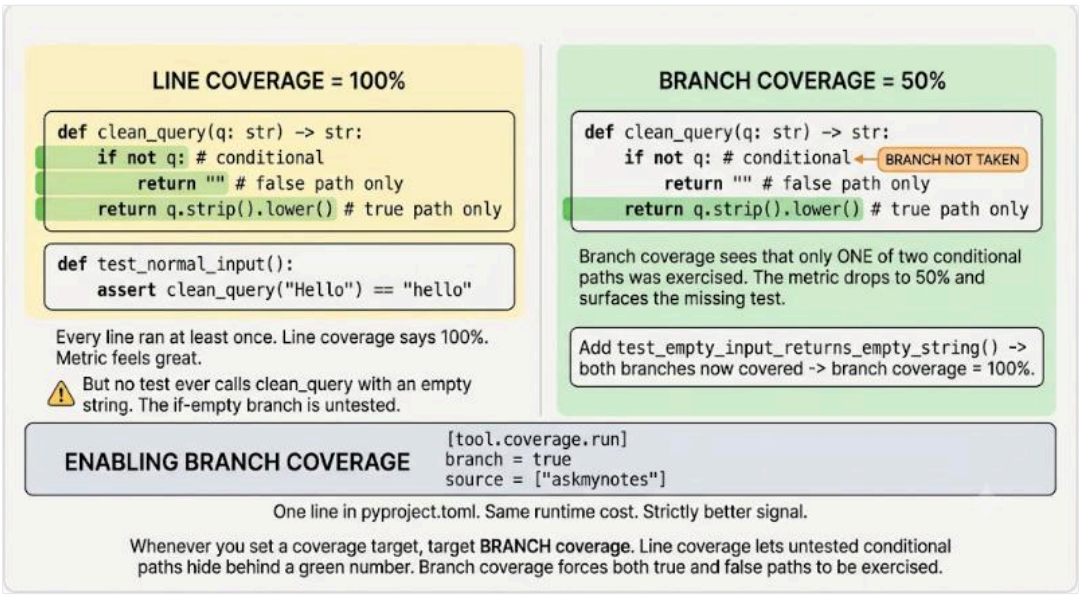
SUMMARY CARD

pytest's power beyond `assert` : **fixtures** for shared setup (parameter-based DI, scoped per-function/module/session, `yield` for teardown, `conftest.py` for cross-file sharing); **parametrize** to run one test across many inputs with clear names per case; **marks** for skipping (`skip` , `skipif` , `xfail`) and selective runs (custom `@pytest.mark.slow`); **mocking** with `unittest.mock` or `mocker` fixture (from pytest-mock) to replace external dependencies (LLMs, APIs, databases). Rule: mock **network + external services + non-determinism**; don't mock your own pure functions.

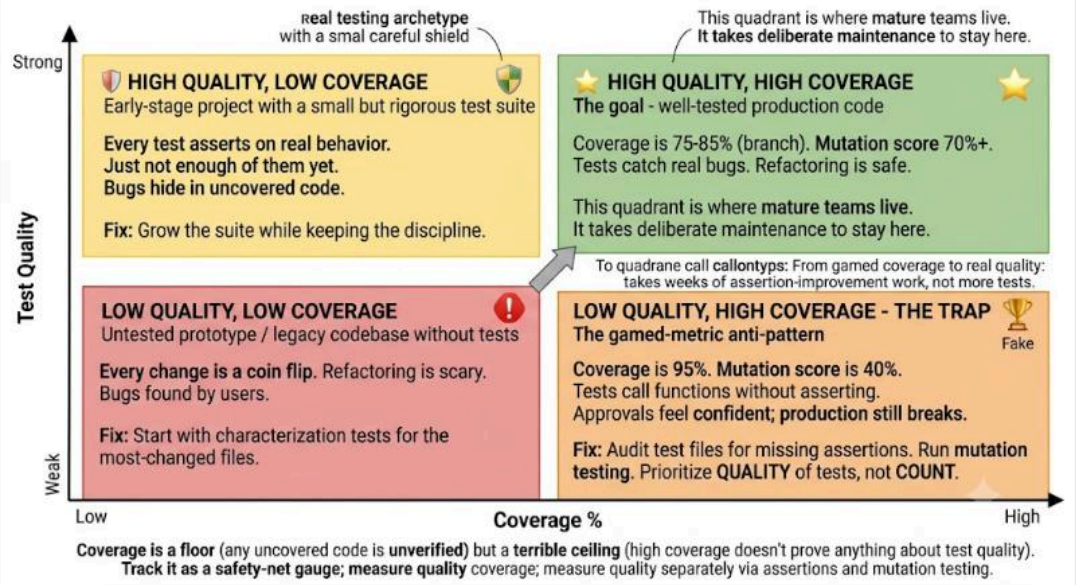
Patch where the name is USED, not where it's DEFINED. `monkeypatch` handles env vars and simple attribute swaps without extra deps.

COVERAGE AND THE TEST PYRAMID - MEASURING WHAT MATTERS

Claude · AI Assistant



Line Coverage vs Branch Coverage - Same Code, Different Rigor



The Coverage-Quality 2x2 - Coverage Number vs Actual Test Value

toml — pyproject.toml - the full coverage + pytest config you'd copy into AskMyNotes

```toml
# pyproject.toml (or setup.cfg / .coveragerc equivalent)

[tool.pytest.ini_options]
minversion = '7.0'
testpaths = ['tests']
python_files = ['test_*.py', '*_test.py']
python_classes = ['Test*']
python_functions = ['test_*']
addopts = [
    '-ra',                          # short summary of all non-passing tests
    '--strict-markers',             # error on unregistered marks (prevents typos)
    '--strict-config',              # error on config typos
    '--tb=short',                   # concise tracebacks
    '--cov=askmynotes',
    '--cov-report=term-missing',    # console output with missing line numbers
    '--cov-report=html:htmlcov',    # HTML report at ./htmlcov/index.html
    '--cov-report=xml',             # for Codecov / GitLab / SonarQube uploads
    '--cov-branch',                 # BRANCH coverage, not just line
    '--cov-fail-under=80',          # CI fails if branch coverage < 80%
]
markers = [
    'slow: marks tests as slow (>1s each)',
    'integration: marks tests requiring external services',
    'gpu: marks tests requiring a GPU',
]
filterwarnings = [
    'error',                        # promote warnings to errors in tests
    'ignore::DeprecationWarning:pkg_resources.*',   # allow this known one
]


[tool.coverage.run]
branch = true
source = ['askmynotes']
omit = [
    '*/tests/*',
    '*/__init__.py',                # trivial
    '*/migrations/*',               # framework-generated
]

[tool.coverage.report]
show_missing = true
skip_covered = false
precision = 1                        # one decimal place
```

```
fail_under = 80
exclude_lines = [
    'pragma: no cover',
    'raise NotImplementedError',
    'if __name__ == .__main__.:',
    'if TYPE_CHECKING:',
    'def __repr__',                 # trivial reprs
    '@abstractmethod',
]

[tool.coverage.html]
directory = 'htmlcov'
title = 'AskMyNotes Coverage'

[tool.coverage.xml]
output = 'coverage.xml'

# ---- Now just run: pytest ----
# All flags come from this file. CI runs the same command as local dev.
```

Student

Everyone talks about 'test coverage'. What does the number actually mean, and what should I aim for?

Claude • AI Assistant

Coverage measures which lines of your production code were executed while your tests ran - nothing more. A high number is not proof of good tests; a low number IS proof of code paths that no test touches at all.

Coverage is best used as a **safety-net gauge** ('any code I haven't covered is code no one has verified') and worst used as a **KPI that engineers optimize** ('let me add tests that call the function without asserting anything to hit 100%'). This subsection covers how to measure coverage, what target makes sense, why 100% is usually a warning sign, and how coverage relates back to the test pyramid.

SETTING UP COVERAGE MEASUREMENT

Student

How do I actually measure it?

Claude • AI Assistant

CONCEPT CARD

For Python: `pytest-cov` is the standard tool (a wrapper around the `coverage` package). Install and run:

bash — Code Example

```
pip install pytest-cov

# Measure coverage over the askmynotes package while running tests
pytest --cov=askmynotes

# Generate an HTML report with per-line highlighting
pytest --cov=askmynotes --cov-report=html
# Open htmlcov/index.html - green = covered, red = uncovered, yellow = partial branches

# Fail CI if coverage drops below a threshold
pytest --cov=askmynotes --cov-fail-under=80
```

Typical output:

code — Code Example

```
---------- coverage: platform linux, python 3.12.0 ----------
Name                        Stmts   Miss  Cover   Missing
---------------------------------------------------------
askmynotes/__init__.py          2      0   100%
askmynotes/chunker.py          15      1    93%   34
askmynotes/loader.py           28      6    79%   45-49, 61
askmynotes/retrieval.py        42     18    57%   22-28, 55-67
---------------------------------------------------------
TOTAL                          87     25    71%
```

The `Missing` column shows exact line numbers not covered - the actionable output. Open those lines in your editor and either write a test that exercises them or delete them as dead code.

LINE VS BRANCH COVERAGE

Student

What's the difference?

Claude • AI Assistant

CONCEPT CARD

Line coverage counts 'did this line run at least once?'. **Branch coverage** counts 'was each branch of every conditional taken?'. Branch is strictly stronger:

python — Code Example

```python
def clean_query(q: str) -> str:
    if not q:                    # LINE covered by tests calling clean_query('hello')
        return ''                # BRANCH not covered - no test with q == ''
    return q.strip().lower()
```

Line coverage says 100% (both lines ran). **Branch coverage says 50%** (the `if` was tested only in the false direction). The branch metric surfaces the missing test for the empty-string case.

Enable branch coverage in `.coveragerc` or `pyproject.toml`:

toml — Code Example

```toml
# pyproject.toml
[tool.coverage.run]
branch = true
source = ['askmynotes']

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 80

[tool.coverage.html]
directory = 'htmlcov'
```

Whenever you set a coverage target, target **branch coverage**, not just line - it costs the same to compute and catches materially more bugs.

WHY 100% IS A RED FLAG

Student

Isn't higher always better?

Claude • AI Assistant

⚠ **WARNING**

No. Hitting 100% almost always means one of three things:

1. **The codebase is trivially small** (fine).
2. **Tests were written to pump the number** - importing modules, calling functions without asserting on the result, testing getters and setters that have no logic. High coverage, low value.
3. **Legitimately unreachable code has been removed or forced-executed** - which means either dead code exists (fine to remove) or tests are contorted to exercise defensive branches that will never fire in production (waste of test-maintenance cost).

A healthy production codebase usually sits at **75-85% branch coverage**. The last 15% is often deliberate: exception paths that only fire on hardware faults, `if __name__ == '__main__'` guards, imports guarded by version checks. Chasing 100% costs more than it saves.

Better target than 100%: 'every merged PR keeps or raises coverage'. That ratchet prevents rot without demanding false perfection.

COVERAGE VS TEST QUALITY

Student

How do I know if my TESTS are good, not just plentiful?

Claude • AI Assistant

CONCEPT CARD

Three checks:

Check 1 - Assertion density: does each test have a specific assertion about the output, or does it just call the function and hope? A test like `def test_chunker(): chunk_text('hello')` executes the function - contributes to coverage - and asserts nothing.

It catches bugs only if the function raises an exception. Grep your test files for tests with no `assert` and audit them.

Check 2 - Mutation testing (the gold standard). Tools like `mutmut` or `cosmic-ray` for Python inject small bugs into your production code ('mutations' - flip `>` to `>=`, replace `+` with `-`, delete a return statement) and rerun your tests.

If your tests still pass despite the injected bug, they didn't actually verify that behavior. Mutation score = % of injected bugs caught by tests.

Aim for 70%+ once your project stabilizes.

Check 3 - Coverage of critical paths. Not all code is equal. The retrieval endpoint of AskMyNotes matters vastly more than a formatter for CLI help text. Weight your test attention accordingly: 95%+ coverage on core paths, less on peripheral tooling.

COVERAGE IN CI

Student

How do teams enforce coverage in CI?

Claude • AI Assistant

CONCEPT CARD

Two patterns:

Pattern 1 - Absolute threshold: 'CI fails if coverage < 80%'. Simple, blunt. Works well early. Downside: legitimate PRs that lower coverage (e.g., adding a defensive branch) get blocked.

Pattern 2 - Delta threshold (better for mature projects): 'CI fails if this PR reduces coverage by more than 0.5%'. Rewards adding tests along with code; allows unavoidable drops with team review.

Tools like Codecov and Coveralls display per-PR coverage delta as a comment on the PR.

yaml — Code Example

```yaml
# .github/workflows/ci.yml snippet (full YAML in 3.5)
- name: Run tests with coverage
  run: pytest --cov=askmynotes --cov-report=xml --cov-fail-under=80
```

```
- name: Upload to Codecov
  uses: codecov/codecov-action@v4
  with:
    files: ./coverage.xml
```

Every PR then shows a coverage report as a bot comment: '+2.3% coverage; 5 new lines covered by tests; 0 uncovered'. Reviewer sees the impact at a glance.

THE PYRAMID REVISITED

Student

How does coverage relate to the test pyramid?

Claude • AI Assistant

CONCEPT CARD

Coverage says nothing about test TYPE - only that lines ran. **A codebase with 90% coverage entirely from slow E2E tests will still have a slow, flaky, hard-to-debug suite** - the ice-cream cone anti-pattern from 3.1.

The healthy shape: **unit tests contribute most of the coverage** (they run fast, isolate bugs precisely), with integration and E2E tests filling gaps for cross-module behavior. If your only coverage comes from E2E tests, unit tests are what's missing - not more of the same E2E.

Practical check: split your coverage report by test type. If unit tests alone give you 70%+ coverage, you're in good shape. If unit tests give you 20% and E2E gives you the rest, the pyramid is inverted.

COVERAGE EXCLUSIONS

Student

What about code I don't want counted?

Claude • AI Assistant

CONCEPT CARD

Some code is legitimately impossible to test cleanly - `if __name__ == '__main__'`, platform-specific branches, defensive `raise NotImplementedError` in abstract methods. Exclude them explicitly:

python — Code Example

```python
if __name__ == '__main__':  # pragma: no cover
    main()
```

or in config:

toml — Code Example

```toml
# pyproject.toml
[tool.coverage.report]
exclude_lines = [
    'pragma: no cover',
    'raise NotImplementedError',
    'if __name__ == .__main__.:',
    'if TYPE_CHECKING:',
]
```

Rule for exclusions: every exclusion should be justified in a comment or PR description. Excluded code is code no test verifies; each one is a small trust debt. A codebase where 30% of code is excluded is not 85% covered - it's 60% covered with an accounting gimmick.

ANALOGY

Coverage is like the **percentage of hallways a burglar-alarm sensor covers**. 100% coverage means every hallway has a sensor - but if the sensors are unplugged (tests without assertions), coverage is theater.

Mutation testing is like **the security firm periodically sneaking a person into your building to check the alarm actually goes off**. Coverage tells you 'sensors installed'; mutation testing tells you 'sensors work'.

Both matter; only one is usually measured.

REAL WORLD APPLICATION

A financial-services team was proud of 97% line coverage. A senior joined, ran mutation testing, and discovered the mutation score was 41% - most of their tests were verifying happy paths but never edge-case bugs.

Six weeks of targeted test improvements (parametrizing edge cases, adding boundary conditions) took mutation score to 78% with coverage barely changing. Line coverage stayed flat; actual bug-catching more than doubled.

Coverage is a floor, not a ceiling.

WARNING

Don't set coverage targets so high that engineers hate them. A 95% target on a 200-file codebase means every PR involves fighting the coverage bot even for tiny changes.

Set the target at what your team can sustain (80% is a common sweet spot), then focus attention on TEST QUALITY, not the number.

COMMON PITFALL / MISTAKE

Excluding files from coverage to hit the target. `# pragma: no cover` on entire modules to make the CI green is coverage laundering - the code still ships, still has bugs, but the number looks good.

Excluded lines should be individual lines with clear justification, not entire files.

EXAM TIP CARD

Interview question: 'What's a good code coverage target?' Confident answer: 'Depends on the codebase. For a production service, 75-85% branch coverage is a healthy sweet spot.

I'd care more about which paths are covered - critical paths at 95%+, peripheral code lower - and about test QUALITY, which coverage doesn't measure. For that I'd use mutation testing periodically.

100% coverage is usually a red flag - either the codebase is trivial or tests are gaming the metric.'

PRACTICE QUIZ

Your coverage report shows 92% line coverage but a mutation test reveals only 45% of injected bugs are caught. What's the most likely problem?

A) The coverage tool is broken

B) Tests execute the code but don't assert on the results

C) Mutation testing is unreliable and should be ignored

D) Coverage should be even higher

▼ Reveal Correct Answer

Answer: B. High coverage + low mutation score is the classic 'tests without assertions' pattern. Code runs (covered) but tests don't check the outputs (mutations survive). Audit tests for missing or weak assertions.

EXERCISE

On AskMyNotes: (a) install `pytest-cov`, (b) run `pytest --cov=askmynotes --cov-report=html --cov-branch`, (c) open `htmlcov/index.html` and identify one file with < 80% coverage, (d) look at the red (uncovered) lines and write one new test that exercises them, (e) rerun and confirm coverage rose. Add `--cov-fail-under=70` to your test command so future drops break CI.

AI DEEP DIVE PROMPT

Ask an AI: '*Given this Python module, generate pytest tests that maximize BRANCH coverage (not just line coverage). Include tests for both true and false paths of every conditional, and tests for every exception path. Do NOT write tests that lack assertions.*'

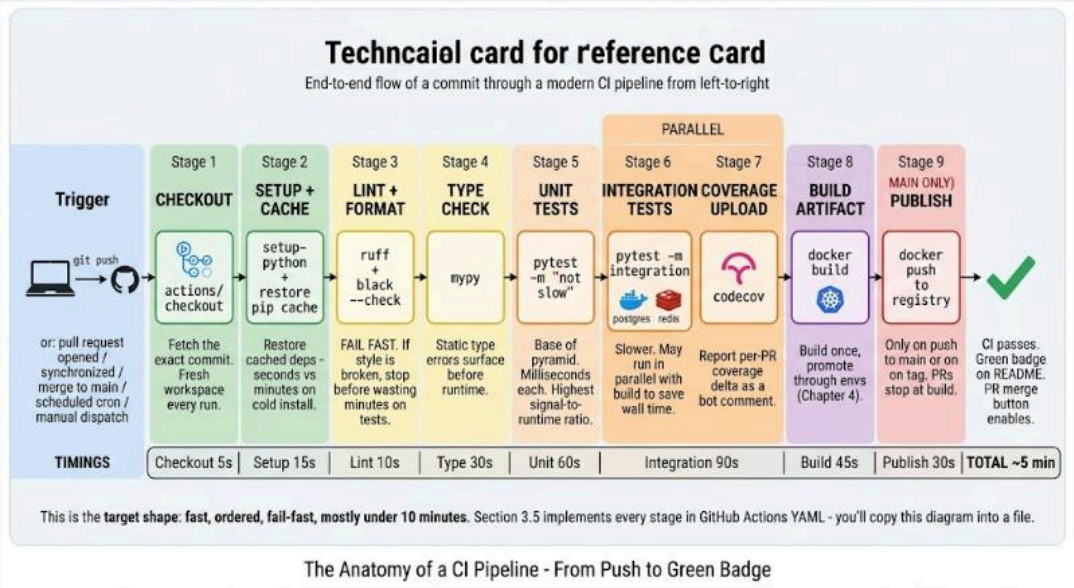
SUMMARY CARD

Coverage measures which lines of production code executed while tests ran. **Line coverage** is easy but weak; **branch coverage** is strictly stronger and worth enabling.

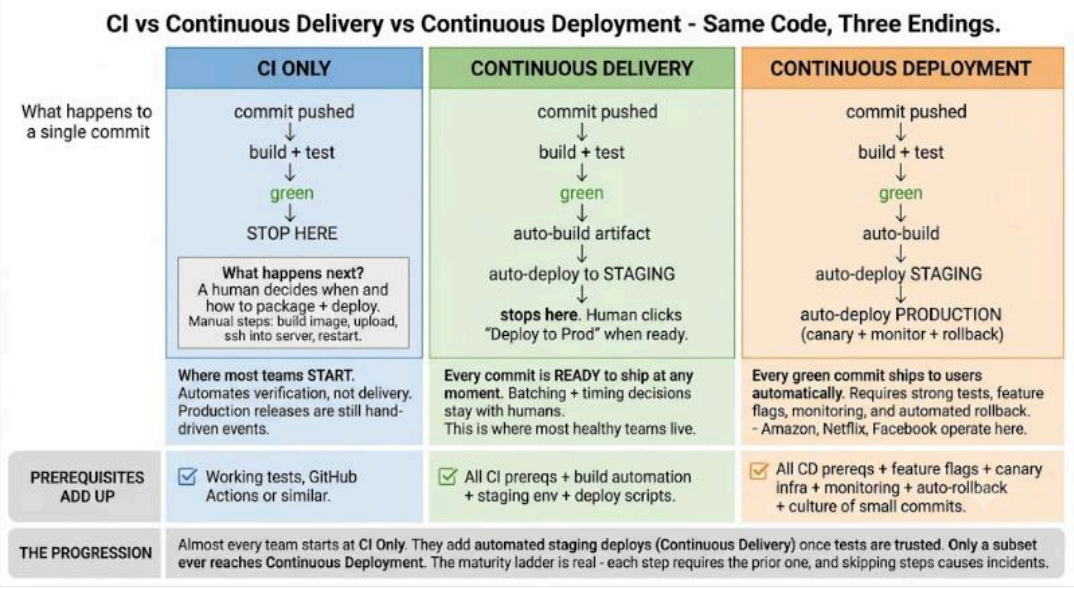
> Healthy target: **75-85% branch coverage** on production code; 100% is a red flag (usually games or dead code). Coverage does NOT measure test QUALITY - use **mutation testing** (mutmut, cosmic-ray) for that gold-standard check.
>
> Enforce coverage in CI via absolute threshold (`--cov-fail-under=80`) or per-PR delta via Codecov. High coverage on E2E-only test suite = ice-cream cone; unit tests should contribute most of the number.
>
> Exclude code sparingly and with justification: excluding files is coverage laundering.

WHAT IS CI/CD - THE AUTOMATION SPINE OF MODERN DELIVERY

Claude · AI Assistant



The Anatomy of a CI Pipeline - From Push to Green Badge



CI vs Continuous Delivery vs Continuous Deployment - Same Code, Three Endings

bash — CI/CD literacy commands - what teams type to interact with pipelines

```
# ---- Local: check what CI will check, before you push ----
ruff check .                          # linter
black --check .                       # formatter (dry-run)
mypy askmynotes/                      # type checker
pytest --cov=askmynotes --cov-branch  # tests + branch coverage
```

```
# Or a one-liner via pre-commit (Chapter 4 sets this up)
pre-commit run --all-files


# ---- GitHub Actions: monitor + control from terminal via gh CLI ----
gh workflow list                        # workflows in this repo
gh run list                             # recent CI runs on any branch
gh run list --branch feature/rerank     # runs on a specific branch
gh run view                             # last run summary
gh run view --log                       # full log output
gh run view --log-failed                # only failed step logs
gh run watch                            # tail the latest run live
gh run rerun <run-id>                   # rerun a failed pipeline
gh run rerun --failed                   # only failed jobs

# Trigger a workflow that supports workflow_dispatch
gh workflow run ci.yml --ref main
gh workflow run deploy.yml -f environment=staging


# ---- Common CI-adjacent conventions on GitHub ----
# README badges (paste at the top of README.md):
# ![CI](https://github.com/<user>/askmynotes/actions/workflows/ci.yml/badge.svg)
# ![Coverage](https://codecov.io/gh/<user>/askmynotes/branch/main/graph/badge.svg)

# Skip CI on a commit that only touches docs (use sparingly)
git commit -m "docs: fix typo [skip ci]"


# ---- Common local aliases teams add to .zshrc / .bashrc ----
alias ci='ruff check . && black --check . && pytest -x'
alias ciw='gh run watch'
alias cif='gh run view --log-failed'
# Then before every push: run 'ci' locally, then push. Saves broken-PR shame.
```

Student

I hear 'CI/CD' constantly. What's the actual concept, and why is every team obsessed with it?

Claude · AI Assistant

Because manual verification is the biggest source of production incidents in real teams. **CI/CD is the automation spine that makes every code change verifiable, packageable, and deployable without a human running steps by hand**.

In this subsection: the definitions (three flavors of CD - Delivery, Deployment, plus GitOps), what a pipeline actually contains, why the practice matters even for solo projects, and the mental model you carry into Section 3.5's hands-on GitHub Actions build. This is the connective tissue between 'I write tests' (Sections 3.1-3.3) and 'my tests run automatically on every PR' (Section 3.5).

THE THREE FLAVORS OF CD

Student

There are THREE CDs?

Claude · AI Assistant

CONCEPT CARD

Yes, and confusing them is a common interview trap:

CI - Continuous Integration: **build + test automatically on every push and every PR**. The purpose: catch integration bugs immediately, before they compound.

Coined by Grady Booch and popularized by XP practices in the late 1990s. Every modern team runs CI; the only variation is HOW (GitHub Actions, GitLab CI, CircleCI, Jenkins, etc.).

Continuous Delivery: CI plus **automated packaging and staging deploy**. Every green commit produces a deployment artifact and lands in a staging environment ready to promote to production - but production deploys require a human click.

The team can ship any commit to production at any moment; they simply choose the moment.

Continuous Deployment: CI plus **automated deploy all the way to production**. Every green commit goes live automatically. Requires strong tests, feature flags, and monitoring. Common at large web/SaaS companies (Amazon deploys thousands of times per day this way).

GitOps (adjacent concept): the Git repository IS the source of truth for production configuration; agents (like Argo CD, Flux) continuously reconcile the running system to match Git. Popular in Kubernetes environments. Deploys happen by merging PRs to config repos.

THE PIPELINE MENTAL MODEL

Student

What's inside a CI pipeline?

Claude • AI Assistant

CONCEPT CARD

A pipeline is an **ordered sequence of stages** that execute on a fresh machine every time. Typical stages for a Python service:

1. **Checkout** - clone the repo, checkout the exact commit under test.
2. **Setup** - install Python, restore cached pip packages (huge speedup).
3. **Install dependencies** - `pip install -e .[dev]` or similar.
4. **Lint / format check** - `ruff check .` and `black --check .` - fails fast on style violations.
5. **Type check** - `mypy askmynotes/` (optional but recommended).
6. **Unit tests** - `pytest -m 'not slow and not integration'` - fast base of pyramid first.
7. **Integration tests** - `pytest -m integration` - may need docker services (postgres, redis).
8. **Coverage upload** - push coverage report to Codecov / etc.
9. **Build artifact** - build the Python wheel or Docker image.
10. **Publish** (on main branch or tag only) - push Docker image to a registry.

Each stage's failure surfaces immediately with the exact log. If lint fails, the pipeline stops - no point running tests on unformatted code. This **fail-fast** ordering saves minutes on every broken PR.

WHY CI MATTERS (EVEN SOLO)

Student

I'm working alone on AskMyNotes. Do I still need CI?

Claude • AI Assistant

CONCEPT CARD

Yes, for four reasons:

1. Environment differences.

Your laptop has certain Python versions, system libraries, and env vars configured. CI runs on a clean VM.

If tests pass on your laptop but fail in CI, it's usually because you unknowingly depended on something local. Better to find that on a PR than during deploy.

2. Discipline forcing. Once CI is set up and shows a green/red badge on your README, you feel bad shipping when it's red. That mild social pressure - visible even in solo projects - dramatically raises code quality.

3. Portfolio signal. A repo with green CI badges and a workflow file signals engineering maturity to anyone (recruiter, teacher, collaborator) reviewing the repo. This is why we insist you set it up in Section 3.5 for AskMyNotes.

4. Preparation for team work. Getting fluent with pipelines on a solo project means you're immediately productive when you join a team that has more elaborate CI/CD. The syntax is the same; only the number of stages grows.

COMMON CI PROVIDERS

Student

What are the popular tools?

Claude • AI Assistant

CONCEPT CARD

Landscape circa 2026:

- **GitHub Actions** - free for public repos, huge free-tier for private, natively integrated. **Default choice for GitHub-hosted projects. This is what we use in 3.5.**
- **GitLab CI/CD** - similar concept, native to GitLab.
- **CircleCI** - independent SaaS; strong on parallelism and orb marketplace.
- **Jenkins** - self-hosted, hugely customizable, older/heavier. Common in enterprise.
- **Buildkite** - self-hosted runners with a hosted control plane; used at many mid/large companies.
- **Argo Workflows / Tekton** - Kubernetes-native pipeline engines.

All share the same core primitives: **triggers** (on push, on PR, on schedule, manually), **jobs** (run on some environment), **steps** (commands within a job), and **artifacts** (outputs preserved across jobs). Learn one and the others feel familiar.

CD SAFETY PATTERNS

Student

Isn't auto-deploying to production scary?

Claude • AI Assistant

CONCEPT CARD

It is if done naively. Real teams deploy safely via layered patterns:

Blue-green deployment: two identical production environments (blue and green). New version deploys to green; traffic gradually shifts. If something breaks, flip back to blue instantly.

Canary deployment: new version rolled out to a small % of production traffic (say, 1%). Metrics watched. If healthy, ramp to 100%. If anomalies detected, roll back automatically.

Feature flags: code path merged and deployed but hidden behind an on/off flag. Team can enable for specific users, then all users, or roll back instantly without redeploying.

Progressive rollout: environment-by-environment (dev -> staging -> prod), with automated tests between each stage.

Automated rollback: monitoring detects error-rate spike after deploy -> pipeline auto-rolls back to the last-known-good release.

Chapter 4 goes deeper on the CD side. For now, the key insight: **CI is universal (every team should have it); CD to production is a maturity ladder (start with staging deploys, add production only when the safety patterns are in place).**

THE THREE GOLDEN SIGNALS OF A HEALTHY PIPELINE

Student

How do teams tell if their CI/CD is 'healthy'?

Claude • AI Assistant

CONCEPT CARD

Three metrics that top teams track:

1. **Pipeline duration** - target < 10 minutes for the fast path (unit + lint + type check). If longer, developers push and context-switch instead of waiting; feedback loop slows.

2. **Flakiness rate** - % of pipeline runs that fail due to nondeterminism, not real bugs. Target near-zero. A 5% flake rate trains the team to re-run instead of investigate, hiding real regressions.
3. **Deploy frequency** - how often new versions reach production.

DORA (DevOps Research and Assessment) research identifies four levels: Low (< monthly), Medium (weekly to monthly), High (daily to weekly), Elite (multiple per day). Higher correlates with lower incident rate, not higher.

Section 3.5 builds a fast, non-flaky CI pipeline that keeps you in the healthy zone from day one.

ANALOGY

CI/CD is the **factory conveyor belt of software delivery**. Before CI, every change was hand-assembled by a person walking parts across the floor - slow, error-prone, unrepeatable.

CI puts every change on the conveyor: automated checks at each station, stopped instantly if anything fails inspection. CD extends the conveyor all the way to shipping.

The point isn't to remove the human - it's to remove the humans from the boring, error-prone bits so they can focus on the interesting decisions (what to build, how to design it).

REAL WORLD APPLICATION

The DORA 'State of DevOps' report, running since 2014, consistently finds that elite performers (multi-daily deploys) have LOWER incident rates and FASTER incident recovery than low performers (monthly deploys). Counter-intuitive but robust: small, frequent, tested changes are safer than rare big-bang releases.

CI/CD is what makes small-and-frequent possible.

WARNING

Do not build a CI pipeline that takes 30 minutes on every push. Developers stop waiting, push and context-switch, and end up rebasing on top of a broken main. Target under 10 minutes for the PR fast path; move slower checks to nightly or post-merge.

COMMON PITFALL / MISTAKE

'Just retry the flaky test' as a team-wide habit. Every flake tolerated is a real bug you'll eventually miss. When a test flakes twice, mark it `@pytest.mark.flaky` and open an issue to fix it - not a retry loop.

EXAM TIP CARD

Interview question: 'What's the difference between Continuous Delivery and Continuous Deployment?' Sharp answer: 'Delivery: every green commit is READY to ship to production but a human clicks the deploy button. Deployment: every green commit ships to production automatically.

Continuous Delivery is a prerequisite; teams add automated production deploys once tests, feature flags, and monitoring are strong enough to justify the risk.'

PRACTICE QUIZ

Your team wants to move from weekly releases to multiple per day. Which capability is MOST critical to build first?

A) A larger QA team to test more releases

B) A trusted automated test suite in CI that gates every merge

C) A rollback runbook the on-call engineer memorizes

D) Nightly integration tests

▼ Reveal Correct Answer

*Answer: B. Frequent deploys are only safe when tests are strong enough to gate merges automatically.

A doesn't scale (humans can't test every commit). C is necessary but useful only when things go wrong; B prevents most of those.

D helps but only for issues caught in integration - unit + integration gating on PR is what enables the frequency.*

EXERCISE

Sketch out the ideal CI pipeline for AskMyNotes on paper: (a) list every stage in order, (b) mark which stages should FAIL FAST (stop the pipeline) and which can run in parallel, (c) estimate the duration of each, (d) mark which stages run on every PR vs only on main vs nightly. Compare to the pipeline you'll implement in 3.5.

AI DEEP DIVE PROMPT

> Ask an AI: '*Explain the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment with a concrete example of what happens to a code change in each model. Then describe the safety patterns (canary, blue-green, feature flags) that make Continuous Deployment safe in production.*'

SUMMARY CARD

CI (Continuous Integration) = build + test on every push; universal in modern teams. **Continuous Delivery** = CI + auto-package + auto-deploy to staging; production requires human click.

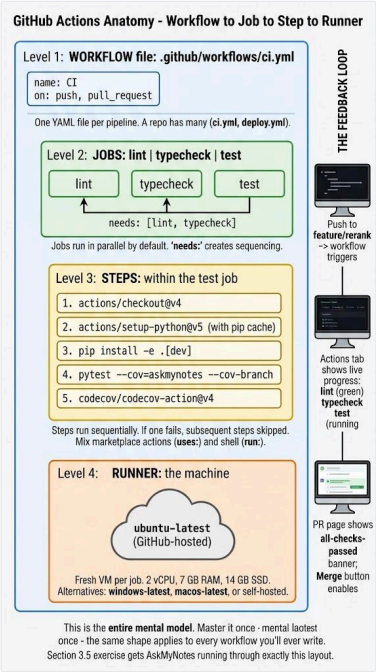
Continuous Deployment = CI + auto-deploy to production; every green commit ships. A **pipeline** is an ordered sequence of stages (checkout -> setup -> lint -> type check -> unit tests -> integration -> build -> publish) running on a clean environment.

Target under 10 minutes for the fast path; keep flakiness near zero. Safety patterns for real production CD: **canary, blue-green, feature flags, automated rollback**.

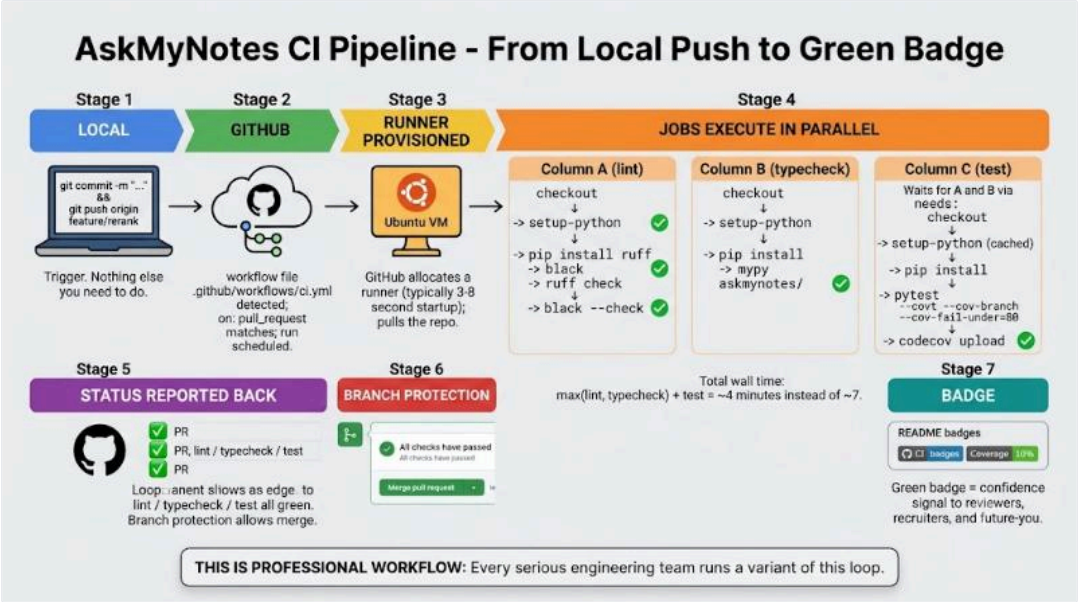
Even solo projects benefit from CI: environment parity, discipline, portfolio signal, team-readiness. Section 3.5 builds this concretely with GitHub Actions.

GITHUB ACTIONS - THE HIGH-EMPHASIS HANDS-ON BUILD

Claude • AI Assistant



GitHub Actions Anatomy - Workflow to Job to Step to Runner



AskMyNotes CI Pipeline - From Local Push to Green Badge

yaml — Production-ready .github/workflows/ci.yml for AskMyNotes - copy this in

```yaml
# .github/workflows/ci.yml
# Runs on every push to main and every PR. Gates merges via branch protection.

name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

# Cancel older runs of this workflow on the same branch when a new push arrives.
# Saves compute on rapid-fire PR updates.
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  # ---------- Fast static checks (parallel) ----------
  lint:
    name: Lint (ruff + black)
    runs-on: ubuntu-latest
    timeout-minutes: 5                # never let a hung job burn compute
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
          cache-dependency-path: 'pyproject.toml'
      - name: Install ruff and black
        run: pip install ruff==0.6.9 black==24.10.0
      - name: Ruff
        run: ruff check .
      - name: Black --check
        run: black --check .

  typecheck:
    name: Type check (mypy)
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
```

```yaml
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: 'pip'
          cache-dependency-path: 'pyproject.toml'
      - name: Install project
        run: |
          python -m pip install --upgrade pip
          pip install -e .[dev]
      - name: mypy
        run: mypy askmynotes/

  # ---------- Tests (matrix across Python versions) ----------
  test:
    name: Test (Py ${{ matrix.python-version }})
    needs: [lint, typecheck]           # skip if fast checks failed
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix:
        python-version: ['3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: 'pip'
          cache-dependency-path: 'pyproject.toml'
      - name: Install project
        run: |
          python -m pip install --upgrade pip
          pip install -e .[dev]
      - name: Run tests with coverage
        env:
          OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
        run: pytest --cov=askmynotes --cov-report=xml --cov-branch --cov-fail-under=80
      - name: Upload coverage to Codecov
        if: matrix.python-version == '3.12'   # upload once, not per matrix cell
        uses: codecov/codecov-action@v4
        with:
          files: ./coverage.xml
          fail_ci_if_error: false
      - name: Upload test artifacts
        if: failure()                         # only on failure - preserve logs
        uses: actions/upload-artifact@v4
        with:
          name: pytest-logs-${{ matrix.python-version }}
          path: |
            .pytest_cache/
            htmlcov/
          retention-days: 7
```

bash — The one-time repo setup that pairs with the workflow above

```bash
# 1) Add the workflow file
mkdir -p .github/workflows
cat > .github/workflows/ci.yml << 'EOF'
# paste the YAML from the previous code block here
EOF

# 2) Add pyproject.toml dev extras so `pip install -e .[dev]` works
# (in pyproject.toml)
#
#   [project.optional-dependencies]
#   dev = [
#       'pytest>=7',
#       'pytest-cov',
```

```
#       'pytest-mock',
#       'ruff',
#       'black',
#       'mypy',
#   ]

# 3) Add CI + coverage badges to README.md (top of file)
cat >> README.md << 'EOF'

![CI](https://github.com/<your-username>/askmynotes/actions/workflows/ci.yml/badge.svg)
![Coverage](https://codecov.io/gh/<your-username>/askmynotes/branch/main/graph/badge.svg)
EOF

# 4) Store OPENAI_API_KEY as an encrypted secret
gh secret set OPENAI_API_KEY --body "sk-your-real-key-here"
# or: GitHub UI -> Settings -> Secrets and variables -> Actions -> New secret

# 5) Commit and push - workflow fires on the resulting PR/push
git checkout -b setup-ci
git add .github/workflows/ci.yml pyproject.toml README.md
git commit -m "ci: add lint + typecheck + test workflow"
git push -u origin setup-ci
gh pr create --base main --title "ci: add CI workflow" \
  --body "Adds lint (ruff+black), typecheck (mypy), test (pytest+cov) jobs"

# 6) Watch the run live
gh run watch

# 7) Once merged to main, turn on branch protection
#    Settings -> Branches -> Branch protection rules -> Add rule for 'main':
#      - Require a pull request before merging
#      - Require status checks to pass before merging
#        Select: Lint (ruff + black), Type check (mypy), Test (Py 3.12)
#      - Require branches to be up to date before merging

# 8) Verify: open a new PR that breaks lint (a super-long unformatted line).
#    The merge button should be DISABLED until lint is fixed.
#    Fix the lint, push, watch the check flip green, merge.
```

Student

Time to actually set up CI. Walk me through GitHub Actions.

Claude · AI Assistant

GitHub Actions is the CI/CD system built into GitHub - workflows are YAML files inside `.github/workflows/` in your repo, and every push, PR, or schedule fires the matching workflow on a fresh Linux/Windows/macOS VM. Free for public repos, generous free tier for private ones, and integrated tightly with PRs and branch protection.

This is Chapter 3's second high-emphasis subsection because writing GitHub Actions workflows is a required skill for every modern software engineering role. By the end of this subsection, AskMyNotes has a green CI badge and every PR runs lint + type check + tests + coverage automatically.

THE MENTAL MODEL

Student

What are workflows made of?

Claude · AI Assistant

CONCEPT CARD

Three nested concepts:

Workflow (`.github/workflows/ci.yml`) - a YAML file with a name, one or more triggers, and one or more jobs. A repo can have any number of workflow files, each for a different purpose (CI, deploy, nightly, release).

Job - a group of steps that run together on one runner (VM). Jobs within a workflow can run **in parallel** by default; they can `needs: <other-job>` to run sequentially instead.

Step - a single command or Action inside a job. Steps run **sequentially**, and if one fails, subsequent steps in that job are skipped by default. A step is either a shell command (`run: pytest`) or a reusable Action from the marketplace (`uses: actions/checkout@v4`).

So the hierarchy is: **workflow -> jobs -> steps**.

YOUR FIRST WORKFLOW

Student

Show me the smallest useful workflow.

Claude • AI Assistant

EXAMPLE CARD

Save this as `.github/workflows/ci.yml` in AskMyNotes and push:

yaml — Code Example

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e .[dev]

      - name: Run tests
        run: pytest -v
```

Push this. Open the **Actions** tab on GitHub - you'll see the run start, watch each step execute in real time, and see a green checkmark (or red X) at the end. That's it. You have CI.

READING A WORKFLOW LINE BY LINE

Student

Break that down.

Claude • AI Assistant

CONCEPT CARD

Line by line:

- `name: CI` - display name in the Actions tab.
- `on: { push: { branches: [main] }, pull_request: { branches: [main] } }` - fires on any push to main AND any PR targeting main. The PR trigger is what gates merges via branch protection.
- `jobs: test:` - one job named 'test' (name is arbitrary).
- `runs-on: ubuntu-latest` - use GitHub's Ubuntu VM. Alternatives: `windows-latest`, `macos-latest`, or `[self-hosted]`.
- `steps:` - the ordered list.
- `uses: actions/checkout@v4` - a marketplace action that checks out the commit under test. `@v4` pins the major version - always pin, never leave it floating.
- `uses: actions/setup-python@v5 with: python-version: '3.12'` - install Python 3.12 on the runner.
- `run: |` starts a multi-line shell command block; the `|` preserves newlines.
- `pip install -e .[dev]` - install the package plus dev extras (requires a `pyproject.toml` with a `[project.optional-dependencies] dev = [...]` section).
- `run: pytest -v` - execute the test suite. Non-zero exit = job fails = CI red.

SPEEDING IT UP WITH CACHING

Student

My workflow takes 3 minutes just to `pip install`. Can I cache?

Claude • AI Assistant

CONCEPT CARD

Yes. The `actions/setup-python` action has built-in caching for pip - one keyword saves 60-90 seconds per run:

yaml — Code Example

```yaml
- name: Set up Python
  uses: actions/setup-python@v5
  with:
    python-version: '3.12'
    cache: 'pip'                          # cache pip downloads
    cache-dependency-path: 'pyproject.toml'
```

The cache key is auto-derived from `pyproject.toml`'s hash - when you change deps, cache is invalidated and rebuilt; otherwise, restore in seconds.

For arbitrary caches (e.g., a downloaded ML model), use `actions/cache` directly:

yaml — Code Example

```yaml
- name: Cache HuggingFace models
  uses: actions/cache@v4
  with:
    path: ~/.cache/huggingface
    key: hf-${{ hashFiles('pyproject.toml') }}
    restore-keys: hf-
```

Caching is the single biggest speedup lever in most CI configurations.

MATRIX BUILDS - TESTING ACROSS PYTHON VERSIONS

Student

How do I test across Python 3.10, 3.11, 3.12 in one workflow?

Claude • AI Assistant

CONCEPT CARD

Use a **matrix**. Add a `strategy:` block that expands one job into N parallel jobs:

yaml — Code Example

```yaml
jobs:
  test:
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false                  # don't cancel siblings if one fails
      matrix:
        os: [ubuntu-latest, macos-latest]
        python-version: ['3.10', '3.11', '3.12']
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
      - run: pip install -e .[dev]
      - run: pytest
```

This expands to 2 x 3 = **6 parallel jobs**. The Actions UI shows each as a separate check ('test (ubuntu-latest, 3.11)'). Great for library authors verifying compatibility; overkill for a solo app.

SECRETS - HANDLING API KEYS

Student

My workflow needs an OPENAI*API*KEY. How do I not commit it?

Claude • AI Assistant

⚠ **WARNING**

Never commit secrets. Use GitHub's encrypted secrets store:

Set the secret: GitHub -> Settings -> Secrets and variables -> Actions -> New repository secret. Add `OPENAI_API_KEY` with your key value.

Reference in workflow:

yaml — Code Example

```yaml
- name: Run integration tests
  env:
    OPENAI_API_KEY: ${{ secrets.OPENAI_API_KEY }}
  run: pytest -m integration
```

GitHub automatically **masks** the secret value in logs - it will show as `***`. Even so, avoid `echo $OPENAI_API_KEY` in workflow steps; the masking is not always perfect against determined leaks.

For deploys: create **environments** (staging, production) with their own separate secret scopes and approval rules. A workflow deploying to `environment: production` can require a human to approve each deploy from the Actions tab.

SPLITTING INTO MULTIPLE JOBS

Student

My workflow has lint + type check + tests. Should they be separate jobs?

Claude • AI Assistant

CONCEPT CARD

Yes, for two reasons: **parallelism** (jobs run concurrently on separate runners, cutting wall time) and **failure isolation** (a lint failure is visible independently of a test failure). Layout:

yaml — Code Example

```yaml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install ruff black
      - run: ruff check .
      - run: black --check .

  typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev]
      - run: mypy askmynotes/

  test:
    runs-on: ubuntu-latest
    needs: [lint, typecheck]              # only run tests if fast checks pass
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12', cache: 'pip' }
      - run: pip install -e .[dev]
      - run: pytest --cov=askmynotes --cov-report=xml --cov-branch
      - name: Upload coverage
        uses: codecov/codecov-action@v4
        with:
          files: ./coverage.xml
```

`lint` and `typecheck` run in parallel; `test` runs only if both pass (via `needs:`). Total wall time = max(lint, typecheck) + test, not sum. On a small repo this saves 30 seconds; on a large one it saves minutes.

BRANCH PROTECTION INTEGRATION

Student

How do I make CI required before merge?

Claude · AI Assistant

CONCEPT CARD

Two steps:

Step 1: Push a workflow that runs on `pull_request: branches: [main]`. Open a PR to trigger it - the check names ('test', 'lint', 'typecheck') will appear in the PR's checks section.

Step 2: GitHub -> Settings -> Branches -> Branch protection rules -> Add rule for `main`:

- Require a pull request before merging (1+ approvals).
- **Require status checks to pass before merging** - then select the specific job names from the dropdown (lint, typecheck, test).
- Require branches to be up to date before merging (rebase before merging).

After this, the Merge button is disabled on any PR where CI hasn't passed. This is the technical enforcement of the discipline discussed in Chapter 2.4.

THE ACTIONS UI - HOW TO DEBUG FAILURES

Student

A CI run failed. What do I look at?

Claude • AI Assistant

CONCEPT CARD

The **Actions tab** shows every run, filterable by workflow / branch / status. Click a failed run to see:

- **Jobs list** (left sidebar) - jobs are colored by status; failed job has a red X.
- **Step-by-step log** for the selected job - click a step to expand its output. The failed step's output is what you need.
- **Summary annotations** - GitHub extracts key errors and pins them at the top ('pytest failed: 3 tests failed').
- **'Re-run failed jobs'** button - useful for flakes (but if you press it more than once, investigate the flake).
- **Artifacts** section (bottom) - download files uploaded during the run (test reports, coverage XML, screenshots).

From the terminal: `gh run list` , `gh run view --log-failed` , `gh run watch` . Full CI monitoring without leaving the shell.

COMPOSITE ACTIONS AND REUSABLE WORKFLOWS

Student

Two of my workflows have identical setup steps. Can I DRY them up?

Claude • AI Assistant

CONCEPT CARD

Two mechanisms:

Composite Action - a reusable set of steps stored in `.github/actions/setup/action.yml` . Referenced as `uses: ./.github/actions/setup` . Good for 'checkout + setup Python + install deps' - the boilerplate at the top of every job.

Reusable Workflow - an entire workflow that other workflows call via `uses: <org>/<repo>/.github/workflows/<file>.yml@<ref>` and `workflow_call: inputs: ...` . Good for standardized deploy pipelines shared across many repos in an org.

For a solo project, ignore these until you have real duplication. For team codebases, both are quality-of-life wins after a few workflows exist.

ANALOGY

A GitHub Actions workflow is like an **automated cooking recipe pinned to your fridge**. `on:` = when to cook ('every time groceries arrive').

`jobs:` = the dishes (main, side, dessert - can cook in parallel). `steps:` = the ordered actions within each dish (chop, sear, plate).

`uses:` = using a pre-mixed sauce from the pantry (marketplace actions). Runners are the kitchen stations.

The green checkmark is 'dinner served'; the red X is 'something burned - check the step logs'.

REAL WORLD APPLICATION

A common pattern at startups: engineering team spends one afternoon setting up a solid CI workflow (lint + type + test + coverage + PR-comment bot). Two months later, defect-escape rate has fallen ~40% and merges take a day less on average.

The investment was under 4 hours; the return compounds indefinitely. This is why every job description for a software engineering role assumes you can write GitHub Actions workflows.

WARNING

Do NOT put secrets or PII in workflow file names, job names, or step names - those appear in public logs. Only the VALUES of `secrets.*` are masked; everything else in the YAML is visible.

COMMON PITFALL / MISTAKE

Pinning actions to `@main` or a floating tag like `@v4` . `@v4` is fine (major-version pinning).

`@main` is dangerous - the action can silently change under you. For maximum safety in security-sensitive workflows, pin to a full SHA: `uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683 # v4.2.2` .

EXAM TIP CARD

Interview question: 'Walk me through a GitHub Actions workflow for a Python project.' A confident answer names the four building blocks (workflow / trigger / job / step), describes a typical CI pipeline (checkout -> setup-python with pip cache -> install -> lint -> typecheck -> test -> coverage upload), mentions parallel jobs with `needs:` for ordering, mentions secrets via `${{ secrets.X }}` , and closes with 'the whole file lives at .github/workflows/ci.yml, is triggered by push and pull_request, and gates merges via branch protection'.

PRACTICE QUIZ

Your workflow's `test` job needs to run AFTER `lint` and `typecheck` both pass. How do you express this?

A) Put all steps in one giant job

B) Use `needs: [lint, typecheck]` on the test job

C) Add a `sleep 30` before pytest

D) Set `runs-on: after-lint-and-typecheck`

▼ Reveal Correct Answer

Answer: B. The `needs:` keyword declares job dependencies; test will only start after both listed jobs succeed. A loses parallelism (lint and typecheck can't run concurrently). C is a race hack. D is not real syntax.

EXERCISE

On AskMyNotes: (a) create `.github/workflows/ci.yml` with the three-job workflow (lint + typecheck + test) shown above, (b) push and confirm the Actions tab shows a green run, (c) enable branch protection requiring the three checks, (d) open a PR that deliberately breaks lint (e.g., add a super-long unformatted line), (e) watch the PR block on the red lint check, (f) fix and see the check flip green + merge button enable, (g) add the CI badge to your README.

AI DEEP DIVE PROMPT

Ask an AI: *'Review my .github/workflows/ci.yml file. Suggest improvements around caching, parallelism, matrix builds, and secret handling. Highlight any anti-patterns like unpinned action versions or missing timeout-minutes. Suggest realistic timings for each stage.'*

SUMMARY CARD

GitHub Actions runs workflows (YAML files in `.github/workflows/`) on triggers (push, pull*request, schedule, workflow*dispatch). Structure: **workflow -> jobs -> steps**.

Jobs parallel by default; `needs:` for sequential. Steps use marketplace **actions** (`uses:`) or shell commands (`run:`).

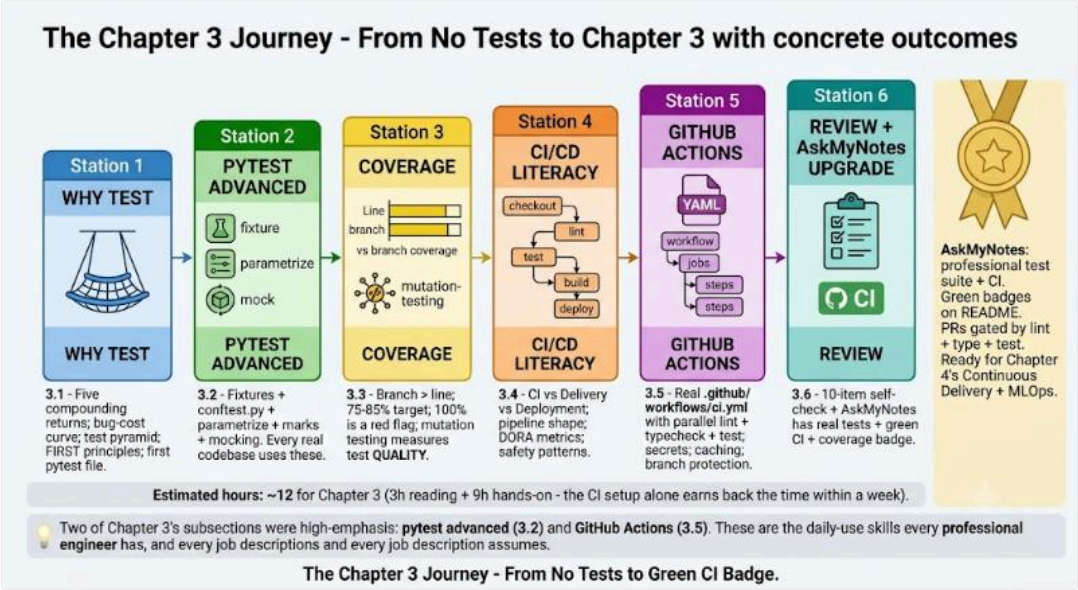
Speed up with `cache: 'pip'` on setup-python; parallelize via multiple jobs; test across configs with `matrix`. **Secrets** live encrypted in GitHub settings, referenced as `${{ secrets.KEY }}`, auto-masked in logs.

Gate merges by requiring specific job names in **branch protection**. Debug failures in the Actions tab (step logs, artifacts, re-run failed).

AskMyNotes now has real CI - green badge on README, tests + lint + typecheck required on every PR.

CHAPTER 3 REVIEW & SUMMARY

Claude • AI Assistant



The Chapter 3 Journey - From No Tests to Green CI Badge

bash — Chapter 3 command reference - the testing + CI toolbox

```
# ---- pytest basics (3.1, 3.2) ----
pytest                              # run all tests
pytest -v                           # verbose - show each test name
pytest -x                           # stop at first failure
pytest --lf                         # rerun last-failed only
pytest -k "chunker"                # only tests matching name pattern
pytest -m "not slow"                # exclude tests marked @pytest.mark.slow
pytest --durations=10               # show 10 slowest tests
pytest -s                           # do not capture stdout (see prints)
pytest --tb=short                   # short tracebacks
pytest -n auto                      # parallel across CPU cores (pytest-xdist)

# ---- Coverage (3.3) ----
pip install pytest-cov
pytest --cov=askmynotes --cov-branch --cov-report=term-missing
pytest --cov=askmynotes --cov-report=html         # HTML at htmlcov/index.html
pytest --cov=askmynotes --cov-fail-under=80       # CI-friendly threshold

# ---- Static checks (3.4) ----
ruff check .                        # lint
ruff check --fix .                  # auto-fix what it can
black .                             # format
black --check .                     # CI dry-run
mypy askmynotes/                    # type check

# ---- GitHub Actions (3.5) via gh CLI ----
gh workflow list                    # workflows in the repo
gh workflow run ci.yml              # trigger workflow_dispatch
gh run list                         # recent runs
gh run view                         # last run summary
gh run view --log-failed            # only failed step logs
gh run watch                        # tail live
gh run rerun --failed               # rerun failed jobs
gh secret set OPENAI_API_KEY        # add encrypted secret from terminal
gh secret list                      # what secrets exist

# ---- Mutation testing (bonus - 3.3 gold standard) ----
pip install mutmut
mutmut run --paths-to-mutate askmynotes/
mutmut results                      # see which mutations survived
mutmut show <id>                    # inspect one surviving mutation
```

CHAPTER SUMMARY

Chapter 3 replaced 'I hope it works' with 'automated proof that it works, on every commit'. **Section 3.1** built the ROI case for testing (five compounding returns, the bug-cost curve, the pyramid, FIRST principles) and wrote our first real pytest file for AskMyNotes' chunker.

Section 3.2 went deep on pytest advanced features - **fixtures** (parameter-injected setup, scopes, `yield` teardown, `conftest.py`), **parametrize** (one test, many inputs, individual failure reporting), **marks** (skip / xfail / custom marks for selective runs), and **mocking** (replacing external APIs and LLMs with fakes to keep tests fast, deterministic, and free). **Section 3.3** covered coverage - line vs branch, why 100% is a red flag, mutation testing as the gold standard for test quality, and how to wire coverage into CI with sensible targets (75-85% branch for production code).

Section 3.4 introduced CI/CD concepts - CI vs Continuous Delivery vs Continuous Deployment, the standard pipeline shape (checkout -> setup -> lint -> type -> unit -> integration -> build -> publish), the DORA metrics (pipeline duration, flakiness, deploy frequency), and the safety patterns (canary, blue-green, feature flags, automated rollback) that make continuous deployment survivable. **Section 3.5** made it real: **GitHub Actions**, the YAML workflow file, the three-level structure (workflow -> jobs -> steps), matrix builds, secrets, caching, and the branch-protection integration that gates PRs on green CI.

AskMyNotes now has a real CI pipeline with green badges on the README.

THE ONE-PAGE MENTAL MODEL

Tests are code that verifies your code, and their ROI compounds because they run automatically forever. The pyramid says: many fast unit tests, some integration, few E2E.

pytest gives you the tools (fixtures, parametrize, marks, mocking) to write clean, focused, isolated tests. Coverage is a floor (any uncovered code is unverified) but not a ceiling (high coverage doesn't prove test quality).

CI = build + test on every push. CD extends to auto-deploy.

GitHub Actions is a YAML file that runs the pipeline on a fresh runner on every trigger, with jobs parallel by default and `needs:` for sequencing. Branch protection makes CI required, not optional.

WHAT'S NEXT

Chapter 4 builds on the CI foundation with the **quality gate workflow** (ruff + black + mypy + pre-commit hooks), then extends into **Continuous Delivery** (build Docker images, push to registry, deploy to staging). Then it pivots to **MLOps** - the practices that treat ML models as first-class deployable artifacts.

You'll learn **MLflow** for experiment tracking (the equivalent of Git for ML training runs) and the **Model Registry** (the equivalent of a Docker registry for trained models). By end of Chapter 4, AskMyNotes has a full CD pipeline plus an MLflow tracking server logging every embedding-model experiment.

SELF-CHECK

Before moving on, verify you can:

1. List the five compounding returns of automated tests and describe the bug-cost curve.
2. Explain the test pyramid and the ice-cream cone anti-pattern.
3. Write a pytest test that follows Arrange / Act / Assert and the FIRST principles.
4. Use a fixture (with `yield` teardown) shared via `conftest.py`.
5. Parametrize a test across multiple input tuples and understand the per-case failure reporting.
6. Mock an external LLM call and assert on the mock's call args.
7. Enable branch coverage in pytest-cov and set a `fail-under` threshold.
8. Explain the difference between CI, Continuous Delivery, and Continuous Deployment.
9. Write a `.github/workflows/ci.yml` with parallel lint + typecheck jobs feeding a test job via `needs:`.
10. Enable branch protection requiring the CI checks to pass before merge.

If any of these are shaky, re-read the relevant subsection. Chapter 4 assumes all ten.

CHAPTER 3 EXERCISE - ASKMYNOTES GETS A REAL TEST SUITE + CI

Complete this before starting Chapter 4:

1. Create `tests/conftest.py` with at least two shared fixtures (sample*text, sample*pdf, mock*openai*client).
2. Write test files for chunker, loader, and retrieval modules - at least 15 tests total, mix of parametrized and mocked.
3. Configure branch coverage in `pyproject.toml` with `--cov-fail-under=70`.
4. Create `.github/workflows/ci.yml` with three jobs: `lint` (ruff + black), `typecheck` (mypy), and `test` (pytest with matrix across Python 3.11 and 3.12, coverage upload to Codecov).
5. Store `OPENAI_API_KEY` as a GitHub secret.

6. Enable branch protection on `main` requiring all three checks + at least one approval + up-to-date branch.
7. Deliberately break the workflow (fail a test, fail lint, fail type check) on a PR and confirm the merge button stays disabled.
8. Add CI and coverage badges to the README top.
9. Optional stretch: add a `nightly.yml` workflow triggered by cron that runs the slow + integration tests hitting the real OpenAI API.

SUMMARY CARD

Chapter 3 built the **automated safety net** for AskMyNotes: **pytest fluency** (fixtures, parametrize, marks, mocking) + **coverage discipline** (branch coverage, 75-85% target, mutation testing awareness) + **CI/CD literacy** (CI vs Delivery vs Deployment, pipeline stages, DORA metrics) + **hands-on GitHub Actions** (workflow YAML, parallel jobs with `needs:`, matrix builds, secrets, caching, branch protection). AskMyNotes now has a green CI badge, tests running on every PR, and merges gated by lint + type check + test success.

That combination is professional-grade quality automation - the baseline for Chapter 4's Continuous Delivery + MLOps.